

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ КОЛЛЕКТИВНОГО ОБМЕНА УЧЕБНЫМИ
МАТЕРИАЛАМИ СТУДЕНТОВ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Студент: _____ / Петров Александр Игоревич, 221–321/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Васильев Денис Борисович, с.п./
подпись *ФИО, уч. звание и степень*

Москва 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Тема ВКР	Веб-приложение для коллективного обмена учебными материалами студентов
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Программа предназначена для организации коллективного обмена учебными материалами между студентами.
Основные функции	1. Регистрация и вход пользователей. 2. Просмотр каталога университетов. 3. Просмотр дисциплин выбранного университета. 4. Просмотр публикаций по выбранной дисциплине. 5. Создание учебных публикаций с текстом и изображениями. 6. Редактирование и удаление собственных публикаций. 7. Просмотр изображений публикации в полноэкранном режиме. 8. Добавление публикаций в избранное и удаление из избранного. 9. Отображение избранных материалов в боковой панели. 10. Публикация и просмотр общих материалов в разделе «Знания». 11. Добавление университетов и дисциплин администратором. 12. Удаление нежелательных публикаций модератором или администратором. 13. Разграничение доступа по ролям: гость, пользователь, модератор, администратор. 14. Безопасное хранение токена авторизации в HTTP-only cookie.
Используемые технологии и платформы	Next.js 16, React 19, TypeScript, Tailwind CSS v4, NestJS, Node.js, PostgreSQL, Prisma ORM, JWT, HTTP-only cookie, Axios, React Hook Form, Multer,

	bcrypt, Sonner, Lucide React, Vitest, React Testing Library, ESLint, Git, Docker, GitHub.
ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Провести анализ предметной области и выявить проблемы существующих способов обмена учебными материалами. 2. Провести обзор аналогов систем обмена учебными материалами и определить их преимущества и недостатки. 3. Определить целевую аудиторию веб-приложения и пользовательские роли. 4. Сформировать функциональные и нефункциональные требования к системе. 5. Спроектировать структуру и навигацию пользовательского интерфейса, подготовить макеты ключевых экранов. 6. Спроектировать архитектуру веб-приложения и клиент-серверное взаимодействие. 7. Разработать модель данных и спроектировать схему базы данных. 8. Реализовать серверную часть веб-приложения. 9. Реализовать клиентскую часть веб-приложения. 10. Провести тестирование разработанной системы и устранить выявленные недостатки. 11. Обеспечить и описать меры информационной безопасности веб-приложения. 12. Описать возможности масштабирования и дальнейшего развития системы.
Состав технической документации	1. Пояснительная записка. 2. Листинг кода: 2 экз.

Состав графической части	1. Презентация. 2. Блок-схема процесса публикации материала. 3. UML-диаграмма последовательности процесса навигации по каталогу и просмотра публикаций. 4. Каркасные макеты пользовательского интерфейса: 4 экз. 5. Изображения интерфейсов приложений-аналогов: 4 экз.
-------------------------------------	---

ПЛАН РАБОТЫ НАД ВКР

Этапы	Недели семестра																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Выбор и уточнение темы ВКР, постановка цели и задач.																		
Анализ предметной области и существующих способов обмена учебными материалами.																		
Обзор аналогов и формирование требований к системе.																		
Проектирование архитектуры веб-приложения и модели данных.																		
Проектирование пользовательского интерфейса и подготовка макетов.																		
Реализация серверной части веб-приложения.																		
Реализация клиентской части веб-приложения.																		
Тестирование, исправление ошибок и описание мер информационной безопасности.																		
Подготовка пояснительной записки, презентации и материалов к защите.																		

РУКОВОДИТЕЛЬ ОП:

«__»____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»____2026, _____ / Васильев Денис Борисович, с.п. /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»____2026, _____ / Петров Александр Игоревич, 221–321/
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: веб-приложение ConSuccess для коллективного обмена учебными материалами студентов и преподавателей.

Цель работы: разработать веб-приложение, предназначенное для централизованного хранения и структурирования учебных материалов по вузам и дисциплинам, обеспечения удобной иерархической навигации по каталогу, формирования персональной базы знаний пользователей с использованием механизма избранного, а также подтверждения статуса преподавателя через заявку с проверкой модератором или администратором.

Объект исследования: веб-приложение, обеспечивающее студентам и преподавателям инструменты для публикации и просмотра учебных материалов в виде публикаций с вложениями, их классификации по образовательным организациям и дисциплинам, навигации по иерархическому каталогу, сохранения материалов в избранное для быстрого доступа, а также управления заявками на получение роли преподавателя.

Предмет исследования: процесс организации коллективного обмена учебными материалами в студенческой среде и автоматизация управления знаниями в формате веб-платформы.

Работа состоит из введения, трех глав, заключения, списка использованных источников и приложения. Общий объем работы составляет 86 страниц. В работе содержится 10 рисунков, 5 таблиц, 2 листинга кода и 1 приложение. Библиография включает 56 источников.

Во введении изложены цель, задачи, объект и предмет исследования, актуальность, новизна и практическая значимость работы. Первая глава посвящена анализу предметной области систем обмена учебными материалами, обзору существующих аналогов, описанию процессов публикации материалов и навигации по каталогу, а также определению требований к системе. Вторая глава описывает проектирование и реализацию веб-приложения: архитектуру клиент-серверного взаимодействия на базе NestJS и Next.js, раз-

работку REST API, проектирование базы данных и пользовательского интерфейса, а также реализацию ключевых модулей: регистрацию и авторизацию, CRUD-операции публикаций материалов с функционалом загрузки вложений, иерархический каталог, раздел «Знания», личные публикации, пометку материалов преподавателя, механизм избранного и обработку заявок на получение роли преподавателя. Третья глава посвящена тестированию системы, вопросам информационной безопасности и авторизации, оценке удобства использования, а также возможностям масштабирования платформы: поддержке нескольких вузов, расширению дисциплин, внедрению полнотекстового поиска, системы жалоб и расширенной модерации. В приложении приведены листинги программного кода. В заключении представлены выводы по выполненной работе и перспективы дальнейшего развития системы.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
1 ПРЕДМЕТНАЯ ОБЛАСТЬ И АНАЛИЗ ТРЕБОВАНИЙ	13
1.1 Анализ предметной и проблемной области	13
1.2 Обзор аналогов и существующих решений	14
1.3 Процесс публикации материала авторизованным пользователем .	21
1.4 Анализ процесса навигации по каталогу и просмотра публикаций	27
1.5 Требования к системе	32
1.6 Выводы по главе	36
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ	37
2.1 Выбор набора технологий	37
2.2 Проектирование модели данных	39
2.3 Проектирование макетов пользовательского интерфейса	41
2.4 Реализация серверной части	46
2.5 Реализация клиентской части	51
2.6 Контейнеризация и локальное развёртывание	59
2.7 Выводы по главе	60
3 ТЕСТИРОВАНИЕ И ОЦЕНКА СИСТЕМЫ	61
3.1 Сценарии функционального тестирования	61
3.2 Информационная безопасность	65
3.3 Масштабируемость и перспективы развития	70
3.4 Выводы по главе	75
ЗАКЛЮЧЕНИЕ	77
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	80
ПРИЛОЖЕНИЕ А	85

ВВЕДЕНИЕ

Современная образовательная среда активно переходит к цифровым форматам обучения и совместной работы. При этом ключевая проблема для студентов сохраняется: учебные материалы часто распределены по разным источникам: чаты, облачные диски, личные заметки, файлы преподавателей. Всё это не имеет единой структуры и быстро теряется. В результате возрастает время на поиск нужной информации, снижается регулярность самостоятельной подготовки и усложняется обмен учебными материалами внутри учебной группы.

Традиционно обмен учебными материалами происходит через неформальные каналы: мессенджеры, социальные сети, общие папки и форумы. Данный подход неудобен из-за отсутствия единого каталога и механизмов классификации, ограниченных возможностей поиска, а также недостаточной прозрачности: сложно понять актуальность файла, к какой дисциплине он относится и насколько он полезен. Отдельной проблемой является отсутствие персонализации: даже если материал найден, его невозможно быстро сохранить «в свою базу» и вернуться к нему позже в несколько кликов.

Актуальность данной работы обусловлена необходимостью разработки веб-приложения, обеспечивающего централизованное хранение, структурирование и быстрый доступ к учебным материалам. Создание системы коллективного обмена учебными материалами позволит упорядочить материалы по вузам и дисциплинам, повысить скорость структурной навигации и повторного использования информации, а также снизить потери учебных ресурсов при смене учебных групп или формата обучения. Кроме того, наличие механизма избранного позволит пользователям формировать персональную базу знаний и поддерживать систематичность подготовки.

Разрабатываемое веб-приложение получило название ConSuccess. Название отражает назначение системы: помогать пользователям успешнее го-

товиться к учебным занятиям за счёт централизованного доступа к материалам и обмена знаниями внутри образовательного сообщества.

Целью данной работы является разработка веб-приложения ConSuccess для коллективного обмена учебными материалами студентов и преподавателей с возможностью структурирования материалов по вузам и дисциплинам, навигации по иерархическому каталогу, сохранения публикаций в избранное и подтверждения статуса преподавателя через проверяемую заявку. Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и выявить проблемы существующих способов обмена учебными материалами.
2. Провести обзор аналогов систем обмена учебными материалами и определить их преимущества и недостатки.
3. Определить целевую аудиторию веб-приложения, пользовательские роли и порядок получения роли преподавателя.
4. Сформировать функциональные и нефункциональные требования к системе.
5. Спроектировать структуру и навигацию пользовательского интерфейса, подготовить макеты ключевых экранов.
6. Спроектировать архитектуру веб-приложения и клиент-серверное взаимодействие.
7. Разработать модель данных и спроектировать схему базы данных.
8. Реализовать серверную часть веб-приложения.
9. Реализовать клиентскую часть веб-приложения.
10. Провести тестирование разработанной системы и устранить выявленные недостатки.
11. Обеспечить и описать меры информационной безопасности веб-приложения.
12. Описать возможности масштабирования и дальнейшего развития системы.

Объект исследования – веб-приложение, предназначенное для организации коллективного обмена учебными материалами.

Предмет исследования – процесс структурирования, хранения и навигации по учебным материалам и поддержка совместного использования знаний студентами в цифровой среде.

Таким образом, разработка веб-приложения позволит централизовать учебные материалы, обеспечить удобную классификацию по вузам и дисциплинам, упростить навигацию и доступ к информации, а также повысить эффективность самостоятельной подготовки за счёт формирования персональной базы знаний через механизм избранного. Отдельный механизм заявок на роль преподавателя повышает доверие к материалам, опубликованным от имени преподавателей, поскольку получение такой роли требует проверки подтверждающего документа. Система может быть востребована в вузах и учебных сообществах, где важно быстрое распространение актуальных материалов и поддержка регулярной учебной практики.

1 ПРЕДМЕТНАЯ ОБЛАСТЬ И АНАЛИЗ ТРЕБОВАНИЙ

1.1 Анализ предметной и проблемной области

Предметная область данного исследования охватывает разработку веб-приложений в сфере цифрового образования, ориентированных на организацию, хранение и распространение учебных материалов между студентами. Данное направление относится к образовательным технологиям и связано с цифровизацией учебного процесса, развитием онлайн-форматов и ростом потребности в удобных инструментах доступа к знаниям [1].

В реальных условиях учебные материалы часто распределены по множеству источников: мессенджеры, социальные сети, облачные хранилища, личные файлы студентов и преподавателей. Такая фрагментация приводит к потере материалов, отсутствию единой структуры и сложностям в поиске. Особенно остро проблема проявляется при подготовке к экзаменам и зачётам, когда требуется быстро восстановить доступ к конспектам, методическим указаниям, лабораторным работам и типовым заданиям по конкретной дисциплине.

Сложившаяся практика обмена учебными материалами через неформальные каналы имеет ряд ограничений. Во-первых, отсутствует удобная классификация: файлы редко систематизируются по вузам и дисциплинам, а названия и содержимое не стандартизированы. Во-вторых, возможности поиска ограничены: даже при наличии общего облачного диска пользователь вынужден просматривать множество папок и файлов. В-третьих, отсутствует персонализация: студент не может быстро сформировать собственный набор полезных материалов и возвращаться к нему без повторного поиска.

Основной проблемой предметной области является отсутствие единой платформы, обеспечивающей структурированное хранение учебных материалов, быструю навигацию по атрибутам: вуз и дисциплина, а также удобные механизмы повторного доступа к найденным материалам. При ручной орга-

низации материалов через папки и чаты неизбежно возникает дублирование, теряется актуальность, снижается качество навигации и возрастает время на подготовку. Это негативно влияет на эффективность самостоятельного обучения и обмена учебными материалами внутри учебного сообщества.

Современные веб-технологии позволяют решать перечисленные задачи за счёт создания централизованных систем управления контентом. Использование клиент-серверной архитектуры, механизмов авторизации, базы данных и инструментов структурной навигации обеспечивает удобный доступ к материалам, поддержку добавления и редактирования контента, а также формирование персональных подборок через механизм избранного [2, 3, 4]. В результате снижается «стоимость нахождения» информации, а доступ к знаниям становится быстрее и более предсказуемым.

Перспективы развития подобных платформ связаны с масштабированием классификаторов: поддержкой нескольких вузов, расширением дисциплин и типов материалов, внедрением модерации контента, системой тегов и рекомендаций, а также анализом популярности материалов. Такие улучшения повышают качество базы знаний и делают обмен учебными материалами более безопасным и удобным для пользователей.

Таким образом, разработка веб-приложения как системы коллективного обмена учебными материалами направлена на решение актуальной задачи: обеспечение структурированного хранения знаний, удобной навигации по каталогу и быстрого повторного доступа к материалам. Это способствует повышению эффективности самостоятельной подготовки, улучшению взаимодействия студентов и упорядочиванию учебной информации в цифровой среде.

1.2 Обзор аналогов и существующих решений

Современный рынок образовательных технологий активно развивается, расширяя спектр цифровых решений для обучения и самообразования.

При этом всё большее значение приобретают сервисы, ориентированные не только на проведение занятий, но и на удобную организацию учебных материалов: их структурирование, быстрый поиск и повторный доступ. Для студентов особенно актуальны инструменты, которые помогают снизить время на «поиск по чатам и папкам» и превращают разрозненные файлы в упорядоченную базу знаний.

Для определения оптимального набора функций и ключевых аспектов, требующих особого внимания при разработке и проектировании веб-приложения, необходимо проанализировать существующие решения, представленные на рынке. В рамках исследования были отобраны наиболее распространённые платформы, используемые студентами для обмена материалами, а также системы, применяемые образовательными организациями для размещения учебного контента. Рассматриваемые решения различаются по назначению и модели использования: от публичных библиотек конспектов и файловых каталогов до корпоративных систем, где материалы размещаются внутри конкретных курсов и доступны ограниченному кругу пользователей.

Анализ аналогов позволяет выявить наиболее удачные практики: каталогизацию материалов по дисциплинам и учебным организациям, развитые механизмы поиска и фильтрации, наличие персональных подборок (избранного), а также возможности управления контентом и контроля качества. Одновременно становятся заметны типовые ограничения: фрагментарность источников, отсутствие единого стандарта описания материалов, сложности с оценкой актуальности и полезности, а также недостаточная персонализация доступа. Таким образом, обзор существующих решений является необходимым этапом для формирования требований к разрабатываемой системе и обоснования проектных решений.

1.2.1 Notion

Notion – универсальная платформа для ведения заметок и построения баз знаний в формате «рабочего пространства». По сути это конструктор страниц, где можно хранить текст, таблицы, файлы и собирать всё в базы данных. Студенты обычно делают в Notion «рабочую тетрадь» на семестр: страницы по дисциплинам, конспекты лекций, чек-листы, ссылки на материалы, иногда – базы «Дисциплины/Темы/Материалы» с фильтрами.

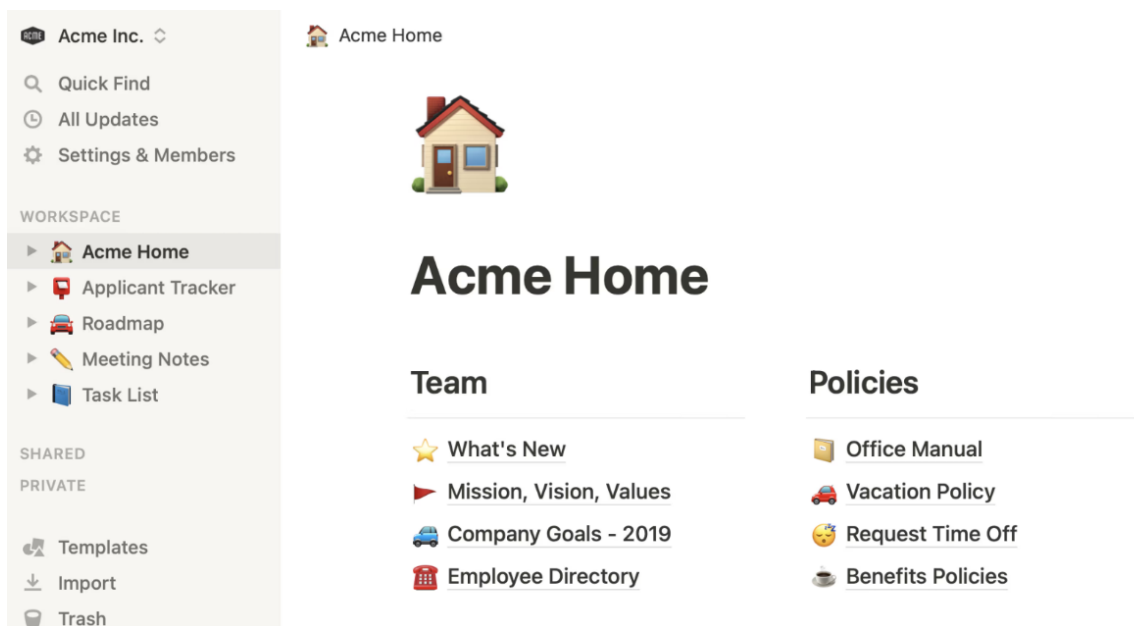


Рисунок 1 – Интерфейс приложения Notion

Релевантные функции для предметной области: древовидные страницы и базы данных с настраиваемыми полями, глобальный поиск по рабочей области, фильтры и представления, совместная работа, хранение вложений и ссылок, шаблоны для стандартизации карточки материала. Сильными сторонами Notion являются гибкая структура и кастомизация, удобный UX базы знаний, поддержка коллективной работы. Ограничения как платформы материалов для студентов: отсутствие доменной модели «вуз – дисциплина – материал» из коробки, ориентация на закрытую рабочую область, отсутствие контроля качества и актуальности, слабая пригодность для массового поиска по множеству вузов и дисциплин, а также избранное на уровне страниц, а не учебных материалов.

1.2.2 YoNote

YoNote – онлайн-платформа для ведения учебных конспектов и организации материалов в цифровом виде. Сервис ориентирован на создание заметок, их структурирование по темам и быстрый доступ к информации в процессе обучения. В типичном сценарии студент использует YoNote как «электронную тетрадь»: создаёт конспекты по дисциплинам, закрепляет важные фрагменты, прикрепляет ссылки или файлы, ведёт списки задач и подготовку к зачётам и экзаменам.

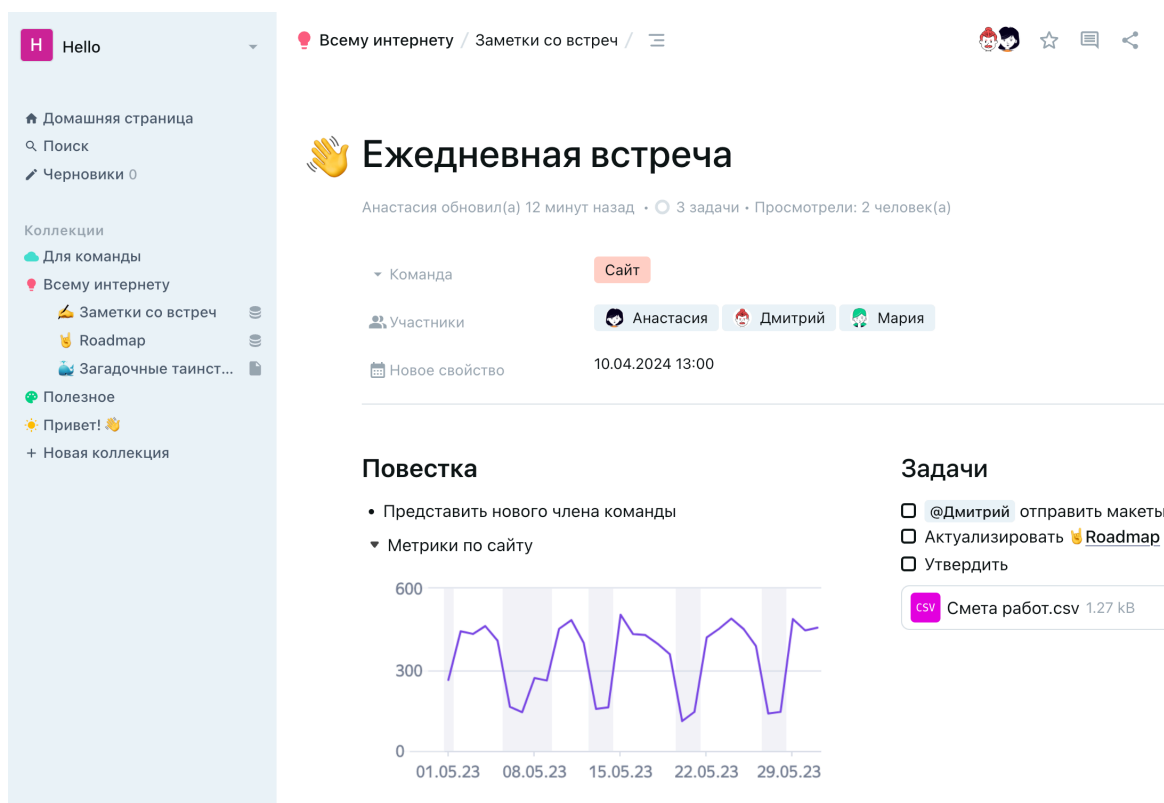


Рисунок 2 – Интерфейс платформы YoNote

Релевантные функции YoNote: создание и ведение заметок, организация материалов по предметам и темам, поиск по заметкам, работа с вложениями и ссылками, доступ с разных устройств, обмен материалами по ссылке или через доступ. Сильные стороны: фокус на учебных сценариях, быстрый доступ к информации, удобство накопления знаний. Ограничения: ориентация на личные заметки, отсутствие строгой доменной модели каталога, ограниченные сценарии массового поиска, отсутствие стандартизированной карточки материала и ограниченная логика избранного.

1.2.3 StuDocu

StuDocu – онлайн-платформа для обмена учебными материалами, где студенты загружают и находят конспекты, краткие конспекты, шпаргалки и материалы для подготовки к экзаменам. Доступ к материалам обычно организован через выбор учебного заведения и курса или дисциплины, чтобы находить документы, релевантные конкретной программе обучения. На платформе также используется модель вознаграждения за публикацию и есть инструменты подготовки на базе загруженных материалов.

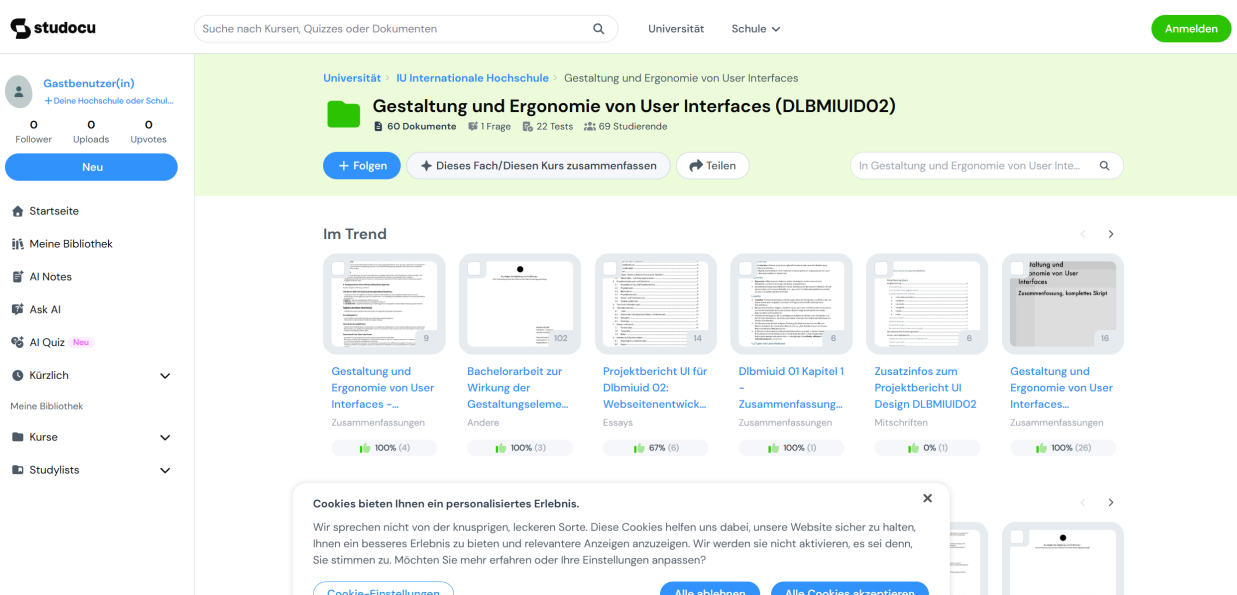


Рисунок 3 – Интерфейс платформы StuDocu

Релевантные функции StuDocu: каталог и поиск по учебному контексту, загрузка учебных документов, предпросмотр и скачивание, механизм мотивации публикации, инструменты подготовки. Сильные стороны: большой объём материалов «от студентов для студентов», ориентация на учебный поиск, низкий порог участия и наличие инструментов подготовки. Ограничения: неоднородное качество контента, риски авторских прав, ограничения доступа через Premium, слабая стандартизация карточки материала.

1.2.4 StudFiles

StudFiles – файловый архив учебных материалов, в котором контент публикуется пользователями и группируется по учебному контексту: вузам и дисциплинам. Платформа ориентирована на хранение и распространение документов: конспектов, методичек, лабораторных, курсовых и других материалов, с возможностью быстрого просмотра и скачивания.

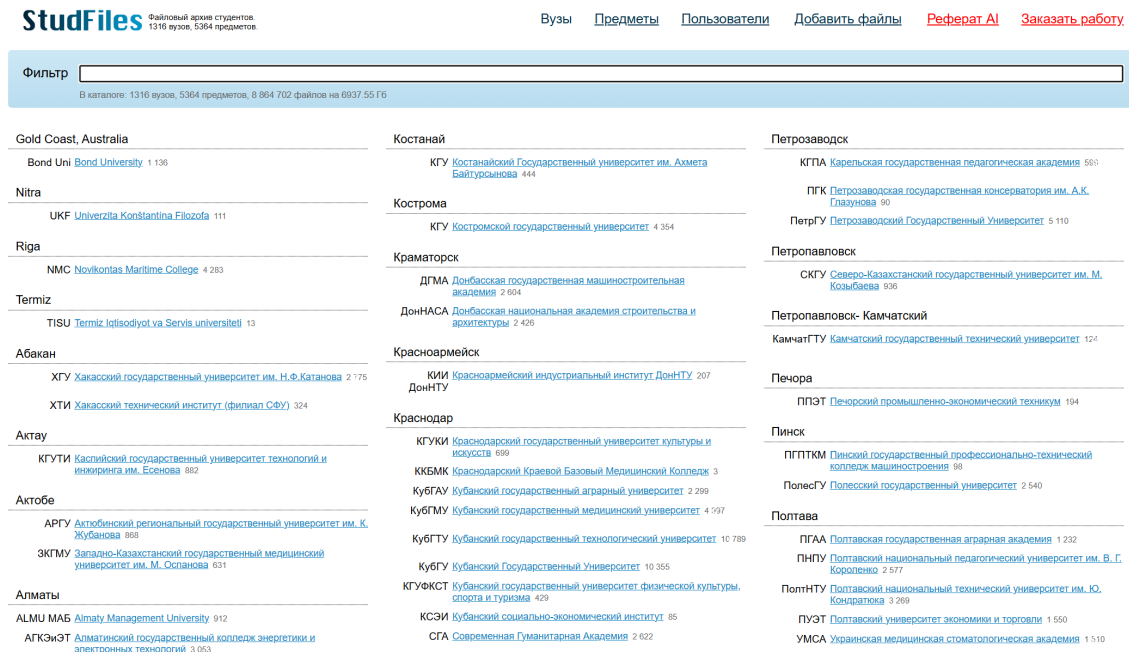


Рисунок 4 – Интерфейс платформы StudFiles

Релевантные функции StudFiles: каталог по вузам и дисциплинам, публикация материалов, поиск и фильтрация, просмотр и скачивание. Сильные стороны: ориентация на учебный контекст, большой объем пользовательских материалов, простой вход в публикацию. Ограничения: неоднородное качество и актуальность, слабая стандартизация описания материалов, ограниченный полнотекстовый поиск, отсутствие явной доменной модели карточки материала.

Таблица 1 – Сравнение аналогов

Критерий	Notion	YoNote	StuDocu	StudFiles
Тип решения	Конструктор рабочей области: страницы и базы данных.	Командная база знаний: документы и представления.	Публичный каталог учебных материалов и инструменты подготовки.	Файловый архив студентов с каталогом по вузам и предметам.
Каталогизация	Нет структуры «вуз – дисциплина» из коробки.	Нет структуры «вуз – дисциплина» из коробки.	Каталог и поиск по университету и курсу или дисциплине.	Каталог «вузы – предметы – файлы».
Публикация материалов	Создание страниц и баз внутри рабочей области; публикация вовне через доступ или ссылку.	Создание документов и баз внутри пространства; обмен через доступ или ссылку.	Загрузка документов пользователями в каталог; доступ может быть частично ограничен.	Загрузка файлов в выбранный предмет; возможна пакетная загрузка.
Поиск и фильтры	Глобальный поиск по рабочей области и фильтры в базах данных.	Поиск по базе знаний и представления баз данных.	Поиск по каталогу и дисциплинам.	Поиск и навигация в каталоге, выбор по вузу и предмету.
Публичность	По умолчанию закрыто: доступ по приглашению, ссылке и правам.	По умолчанию закрыто: доступ по правам или ссылке.	Публичная библиотека материалов.	Публичный архив через каталог сайта.
Совместная работа	Совместное редактирование, комментарии, права доступа.	Совместное редактирование и комментарии.	Коллективность через публикацию и потребление.	Коллективность через публикацию и потребление; совместного редактирования нет.

Продолжение таблицы 1

Критерий	Notion	YoNote	StuDocu	StudFiles
Избранное	Закладки и избранные страницы, не отдельное избранное материалов.	Закладки и быстрый доступ к документам.	Сохранение материалов как часть работы с библиотекой.	Закладки на уровне материалов без развитой каталожной логики.
Контроль качества и модерация	Контроль через права команды; нет модерации публичного каталога пользовательских материалов.	Контроль через права и регламенты команды.	Проверка материалов и правила публикации; качество зависит от автора.	Качество зависит от пользователей; возможны дубли и устаревшие версии.
Мотивация публикации	Нет встроенной награды; мотивация – удобство команды или личная организация.	Нет встроенной награды; мотивация – работа внутри команды.	Награды за принятые загрузки.	Обычно без явной награды; мотивация – обмен и доступ к архиву.

Вывод по сравнению аналогов состоит в том, что ни один из рассмотренных сервисов не сочетает публичный иерархический каталог «университет – дисциплина – публикация», поддержку вложений разных типов, механизм избранного, личные материалы и оценку полезности публикаций. Поэтому разрабатываемая система должна перенять удобные элементы аналогов: карточную модель материала, структурную навигацию и быстрый доступ к сохранённым материалам, но реализовать собственную строгую доменную структуру, ориентированную на студенческий обмен учебными материалами.

1.3 Процесс публикации материала авторизованным пользователем

Процесс публикации материала авторизованным пользователем представляет собой последовательность действий, направленных на создание новой публикации с возможностью добавления вложений. В отличие от неформального обмена материалами через мессенджеры и облачные хранилища, публикация в системе структурирует контент по иерархии «университет –

дисциплина – публикация», обеспечивает постоянный доступ к публичным материалам для всех пользователей и поддерживает прикрепление изображений, PDF-документов и MP4-видео непосредственно к публикации.

В рамках данного процесса рассматривается пользователь, уже прошедший вход в систему. Поэтому отдельный переход к странице авторизации не входит в алгоритм публикации; проверка токена выполняется на серверной стороне как технический контроль корректности запроса.

В системе поддерживается два типа публикаций. Первый тип – тематические публикации, привязанные к конкретной дисциплине университета и образующие структурированный каталог. Второй тип – публикации раздела «Знания», не привязанные к конкретному университету или дисциплине; они образуют общую базу статей и заметок, доступных всем пользователям.

1.3.1 Участники процесса и роли

В процессе публикации материала участвуют следующие роли:

1. Авторизованный пользователь (студент или преподаватель) – создаёт публикацию: открывает дисциплину или раздел «Знания», заполняет заголовки, текст, выбирает режим доступа и при необходимости прикрепляет до пяти вложений. Пользователь с ролью преподавателя обладает теми же правами, что и студент, но его публикации дополнительно помечаются как материалы преподавателя.

2. Клиентская часть веб-приложения – открывает форму создания публикации, выполняет первичную валидацию данных, отправляет форму на сервер через API и информирует пользователя об ошибках.

3. Серверная часть веб-приложения – проверяет токен, типы файлов и лимиты публикаций, сохраняет файлы, создаёт записи в базе данных и публикует материал на сервере.

1.3.2 Входные данные и выходные результаты процесса

Входными данными процесса публикации являются:

1. Заголовок публикации – краткое название, отражающее содержание материала.
2. Основной текст – развёрнутое содержание публикации: конспект, описание, методика и т. п.
3. Вложения – до пяти файлов: изображения, PDF-документы и MP4-видео.
4. Режим доступа – публичная публикация для общего каталога или личная публикация, доступная автору в отдельном разделе.
5. Навигационный контекст – идентификаторы университета и дисциплины для тематических публикаций либо раздел «Знания».

Выходными результатами процесса являются:

1. Новая запись публикации в базе данных с привязкой к дисциплине или разделу «Знания», автору и дате создания.
2. Записи вложений для каждого загруженного файла с URL-путём к файлу на сервере.
3. Доступность публичной публикации для просмотра всеми пользователями в соответствующем разделе каталога либо доступность личной публикации только автору.

1.3.3 Этап 1. Открытие раздела и формы публикации

Процесс начинается с того, что авторизованный пользователь открывает нужную дисциплину или раздел «Знания». Затем он нажимает кнопку «Добавить публикацию», после чего клиентская часть приложения открывает форму создания публикации.

1.3.4 Этап 2. Заполнение формы публикации

Ключевым отличием системы от обмена файлами через чат является обязательное структурирование данных. Пользователь заполняет:

1. Заголовок. Название должно кратко и информативно отражать содержание публикации, например: «Лекция 3: алгоритмы сортировки», «Решения типовых задач по теореме».

2. Текст публикации. Развёрнутое содержание: конспект, методические указания, разбор задач и т. п.

3. Вложения. Пользователь может прикрепить до пяти файлов: изображения со схемами и сканами, PDF-документы с методическими материалами или MP4-видео с разбором задания. Система принимает только разрешённые типы файлов.

4. Признак личной публикации. Если пользователь отмечает публикацию как личную, она не попадает в общий список материалов дисциплины и отображается автору в разделе «Мои личные», рядом с которым выводится количество личных материалов по выбранной дисциплине.

Форма выполняет клиентскую валидацию: проверяет заполненность обязательных полей и корректность выбранных вложений. Если данные формы некорректны, пользователь получает сообщение об ошибке на клиентской стороне и возвращается к редактированию полей формы.

1.3.5 Этап 3. Отправка данных на сервер

Если данные формы прошли клиентскую проверку, пользователь нажимает кнопку «Добавить». Данные отправляются на сервер через API в формате `multipart/form-data`. Next.js API-маршрут передаёт запрос серверной части NestJS: используется маршрут `POST /subjects/:subjectId/posts` или аналогичный маршрут для раздела «Знания». Тело запроса передаётся как `Buffer` с исходным заголовком `Content-Type`, что сохраняет корректную `multipart`-границу.

1.3.6 Этап 4. Серверная проверка и публикация материала

На сервере выполняется следующая последовательность действий:

1. AuthGuard проверяет JWT-токен из заголовка `Authorization` и извлекает данные пользователя.
2. `FilesInterceptor (multer)` обрабатывает прикрепленные файлы: проверяет допустимость типа файла, для изображений использует MIME-тип, для PDF-документов и MP4-видео – MIME-тип и расширение, сохраняет файлы в папку `uploads/` с именем вида `{timestamp}-{random}.{ext}`.
3. `PostsService` проверяет ограничения на количество публикаций: не более десяти публикаций в день и не более пятидесяти публикаций всего для одного пользователя.
4. `PostsService` создаёт запись `Post` в базе данных: заголовок, текст, идентификаторы дисциплины и пользователя, дату создания и признак личной публикации.
5. Для каждого загруженного файла создаётся запись `Attachment` с URL вида `/uploads/имя-файла.расширение`, связанная с созданной публикацией.
6. При успешной проверке сервер публикует материал: сохраняет файлы и создаёт связанные записи публикации и вложений.

Если токен, файлы или лимиты не проходят проверку, сервер возвращает ошибку, а клиентская часть информирует пользователя и оставляет его на форме для исправления данных. Если проверка успешна, материал публикуется на сервере и становится доступен согласно выбранному режиму доступа.

1.3.7 Этап 5. Обработка ошибок

В процессе публикации возможны следующие ошибки:

1. Ошибки клиентской валидации: пустые обязательные поля или некорректно выбранные вложения. Пользователь получает сообщение на клиентской стороне и возвращается к заполнению формы.

2. Ошибка проверки токена: сервер не подтверждает корректность запроса, возвращает ошибку, а клиентская часть выводит уведомление пользователю.

3. Недопустимый тип файла: файл не относится к разрешённым изображениям, PDF-документам или MP4-видео, сервер отклоняет файл и возвращает ошибку.

4. Превышение лимита публикаций: пользователь пытается создать более десяти публикаций за день или более пятидесяти публикаций всего.

5. Сбой загрузки: ошибка сети или сервера, вывод уведомления на клиентской стороне; введённые данные сохраняются в форме для повторной отправки.

Таким образом, процесс публикации материала авторизованным пользователем включает: открытие дисциплины или раздела «Знания», открытие формы создания публикации, заполнение заголовка, текста, вложений и режима доступа, клиентскую проверку данных, отправку формы на сервер через API, серверную проверку токена, файлов и лимитов, а также публикацию материала на сервере. При ошибках пользователь получает сообщение на клиентской стороне и возвращается к заполнению формы. Иерархическая привязка «университет – дисциплина – публикация» обеспечивает структурированный доступ к материалам, а поддержка изображений, PDF-документов и видео позволяет публиковать более полный учебный контент: конспекты, схемы, методические файлы и разборы задач.

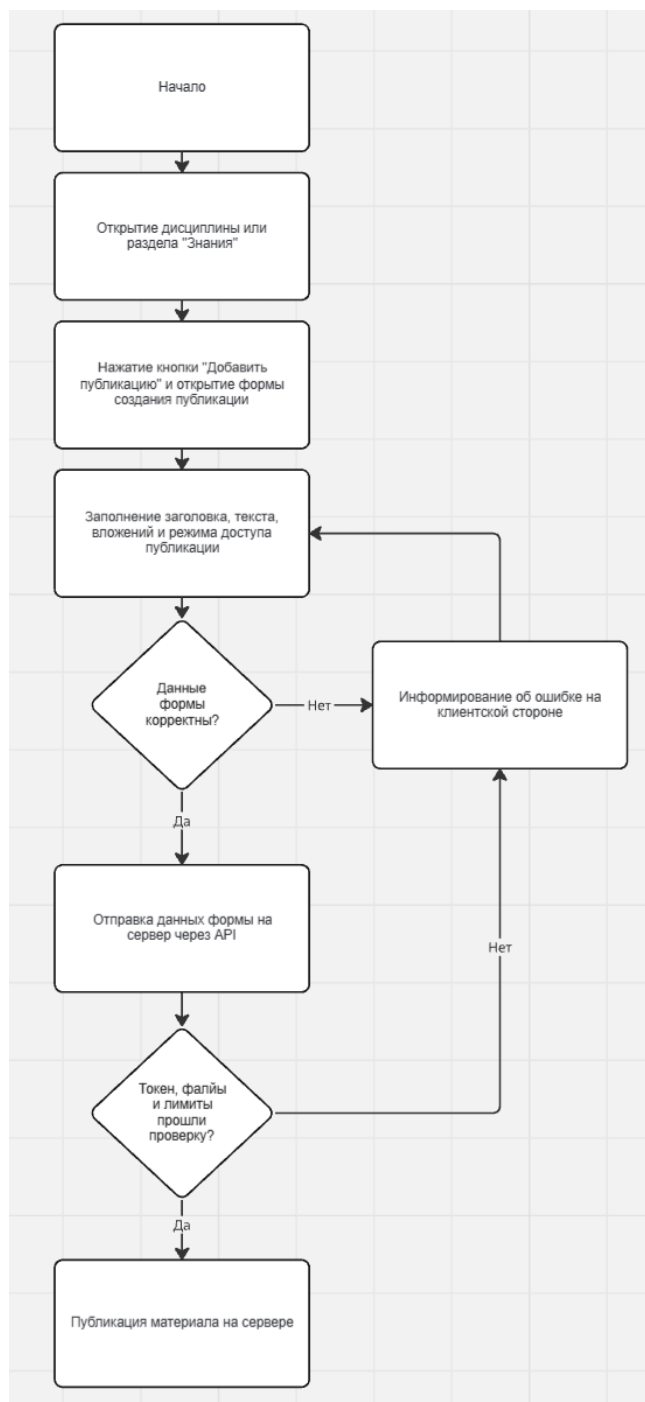


Рисунок 5 – Блок-схема процесса публикации материала авторизованным пользователем

1.4 Анализ процесса навигации по каталогу и просмотра публикаций

Процесс навигации по каталогу и просмотра публикаций в системе направлен на предоставление пользователям удобного и предсказуемого доступа к учебным материалам через иерархическую структуру. Система исполь-

зует подход структурной навигации: каталог организован по принципу «университет – дисциплина – публикации», что позволяет пользователям последовательно сужать область просмотра до релевантного учебного контекста без необходимости вводить поисковые запросы.

Помимо основной иерархии, в системе предусмотрен отдельный раздел «Знания», агрегирующий публикации общего характера, не привязанные к конкретному университету или дисциплине. Дополнительными механизмами доступа к материалам являются система избранного, позволяющая пользователям сохранять нужные публикации, раздел личных материалов по дисциплине и отметки «Нравится», помогающие оценивать полезность публикаций.

1.4.1 Участники процесса и роли

В процессе навигации и просмотра участвуют следующие роли:

1. Пользователь, авторизованный или гость, – просматривает каталог, переходит по уровням иерархии, открывает страницы публичных публикаций; авторизованный пользователь добавляет доступные ему публикации в избранное, отмечает их как понравившиеся, просматривает собственные личные материалы и при необходимости отправляет заявку на получение роли преподавателя. Если автор публикации имеет роль преподавателя, интерфейс отображает соответствующую пометку.

2. Система (веб-приложение) – обрабатывает запросы к серверным компонентам, получает данные через API, формирует страницы с карточками и деталями публикаций.

3. Администратор или модератор – может редактировать или удалять нежелательный контент, что влияет на состав каталога.

1.4.2 Структура каталога и уровни навигации

Основная навигация в системе строится по трём уровням иерархии:

1. Уровень 1 – список университетов (/universities). Отображает карточки всех доступных университетов: логотип, название, город. Пользователь выбирает нужный университет для перехода к его дисциплинам.

2. Уровень 2 – дисциплины университета (/universities/[id]). Отображает карточки дисциплин, прикрепленных к выбранному университету. Пользователь выбирает дисциплину для просмотра публикаций.

3. Уровень 3 – публикации дисциплины. Маршрут имеет вид /universities/[id]/ subjects/[subjectId]. Отображает список публичных публикаций в виде карточек: заголовок, краткий предпросмотр текста, первое вложение при наличии и счётчик отметок «Нравится». Пользователь открывает конкретную публикацию для просмотра полного содержания.

Дополнительно доступен раздел «Знания» – список общих публикаций, не привязанных к конкретной дисциплине. Для авторизованного пользователя на странице дисциплины также доступен переход к разделу «Мои личные», где отображаются его личные материалы по выбранному предмету. Главная страница системы отображает публичные публикации из разных разделов, обеспечивая быстрый доступ к актуальным и полезным материалам.

1.4.3 Входные данные и выходные результаты процесса

Входными данными процесса навигации являются:

1. Действия пользователя – переходы по ссылкам, нажатие кнопок карточек, клики по кнопке избранного, отметке «Нравится» и переходу к личным материалам.

2. Параметры маршрута – идентификаторы университета, дисциплины, публикации в URL-адресе страницы.

3. Токен авторизации – определяет доступность кнопки добавления в избранное, отметки «Нравится», личных публикаций, формы заявки на роль

преподавателя и отображение действий с публикацией: редактирование, удаление, изменение режима доступа.

Выходными результатами процесса являются:

1. Список карточек: университеты, дисциплины или публикации, на соответствующем уровне каталога.
2. Полная страница публикации с текстом, вложениями, данными об авторе, отметками «Нравится» и действиями, доступными текущему пользователю.
3. Актуальный список избранного в боковой панели, а для авторизованного пользователя – состояние его заявки на роль преподавателя при переходе в соответствующий раздел.

1.4.4 Этапы навигации

Пользователь переходит на страницу списка университетов. Серверный компонент запрашивает данные через API и отображает карточки университетов в сетке. Пользователь выбирает нужный университет и переходит к списку его дисциплин. Кнопка «Назад» обеспечивает возврат к предыдущему уровню.

На странице университета отображаются карточки всех дисциплин данного учебного заведения. Пользователь выбирает нужную дисциплину и переходит к списку публикаций по ней.

На странице дисциплины отображаются карточки публичных публикаций. Каждая карточка содержит заголовок публикации, краткий предпросмотр текста, первое прикреплённое вложение при наличии и ссылку «Открыть» для перехода к полному тексту. Если пользователь авторизован, рядом с кнопкой добавления публикации отображается кнопка «Мои личные» с числом личных материалов текущего пользователя по данной дисциплине.

Страница публикации содержит полный текст материала, заголовок, данные об авторе, дату создания и вложения. Если публикация создана поль-

зователем с ролью преподавателя, рядом с данными автора выводится пометка «Материал от преподавателя». Галерея реализована как клиентский компонент с возможностью открытия изображения или видео в полноэкранном режиме; PDF-документы открываются в отдельной вкладке. Автору публикации доступны кнопки редактирования, удаления и изменения режима доступа; пользователям с ролями MODERATOR и ADMIN доступны действия редактирования и удаления любых публикаций. Кнопка «Назад» возвращает пользователя к списку публикаций дисциплины.

На карточке и странице каждой публикации отображается количество отметок «Нравится». Авторизованный пользователь может поставить или убрать такую отметку, а также добавить публикацию в избранное через кнопку-звёздочку. Все сохранённые публикации, включая собственные личные материалы пользователя, отображаются в боковой панели «Избранное», доступной на всех страницах авторизованной зоны, в виде списка ссылок. Личные публикации в избранном дополнительно помечаются меткой «Личный». Это позволяет пользователям формировать персональную подборку нужных материалов и возвращаться к ним без повторной навигации по каталогу.

При навигации возможны следующие ошибки: несуществующий идентификатор университета, дисциплины или публикации приводит к странице 404; личная публикация недоступна постороннему пользователю и также возвращает 404; ошибка сети или недоступности API отображается соответствующим сообщением; попытка выполнить защищённое действие без авторизации приводит к перенаправлению на страницу входа.

Таким образом, навигация по каталогу строится по принципу иерархического сужения: пользователь последовательно переходит от списка университетов к дисциплинам и затем к публикациям по конкретному предмету. Раздел «Знания» обеспечивает доступ к материалам общего характера, личные публикации позволяют автору хранить собственные материалы внутри той же предметной структуры, а механизм избранного позволяет формиро-

вать персональную подборку для быстрого повторного доступа. Данный подход обеспечивает предсказуемую навигацию и соответствует привычной модели обучения, организованной по учебным заведениям и дисциплинам.

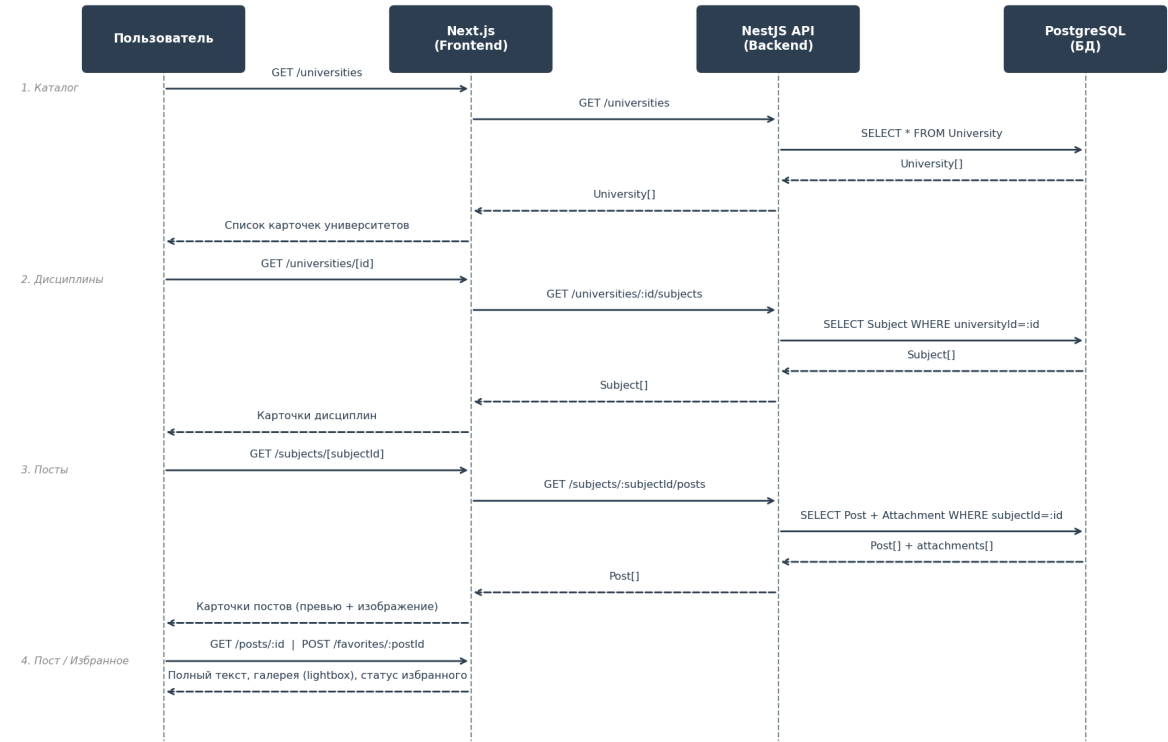


Рисунок 6 – UML-диаграмма последовательности процесса навигации по каталогу и просмотра публикаций

1.5 Требования к системе

На основе анализа предметной области, целевой аудитории и обзора аналогов сформулированы функциональные требования к разрабатываемому веб-приложению, а также определён критический путь для каждой пользовательской роли.

1.5.1 Функциональные требования

1. Система должна обеспечивать регистрацию пользователя по уникальному имени пользователя и паролю.
2. Система должна обеспечивать аутентификацию пользователя с хранением JWT в HTTP-only cookie и возможностью выхода из аккаунта.

3. Любой пользователь, в том числе гость, должен иметь возможность просматривать каталог университетов, дисциплин и публикаций без авторизации.

4. Авторизованный пользователь должен иметь возможность создать публикацию с заголовком, текстом и до пяти вложений в рамках выбранной дисциплины.

5. Авторизованный пользователь должен иметь возможность публиковать материалы в разделе «Знания» без привязки к конкретному университету или дисциплине.

6. Автор публикации должен иметь возможность редактировать заголовки и текст, управлять вложениями, изменять режим доступа и удалять собственные публикации.

7. Авторизованный пользователь должен иметь возможность создавать личные публикации и просматривать их в отдельном разделе дисциплины; кнопка перехода к разделу должна отображать количество личных публикаций текущего пользователя.

8. Авторизованный пользователь должен иметь возможность добавлять доступные ему публикации в избранное и убирать их оттуда; список избранного отображается в постоянной боковой панели и включает собственные личные публикации пользователя.

9. Авторизованный пользователь должен иметь возможность ставить и убирать отметку «Нравится» у доступных ему публикаций.

10. Страница публикации должна отображать прикрепленные изображения, PDF-документы и MP4-видео с корректным способом просмотра для каждого типа файла.

11. Система должна поддерживать роль TEACHER с правами обычного пользователя и помечать публикации таких авторов как материалы преподавателя.

12. Авторизованный пользователь с ролью USER должен иметь возможность отправить заявку на получение роли преподавателя, указав ФИО и загрузив одно изображение подтверждающего документа.

13. Пользователи с ролями MODERATOR и ADMIN должны иметь возможность просматривать заявки на получение роли преподавателя, видеть ФИО и изображение документа, а также одобрять заявку с автоматическим назначением пользователю роли TEACHER.

14. Пользователь с ролью ADMIN должен иметь возможность назначать выбранному пользователю роль MODERATOR.

15. Система должна ограничивать создание публикаций: не более десяти публикаций в день и не более пятидесяти публикаций всего на одного пользователя.

16. Пользователи с ролями MODERATOR и ADMIN должны иметь возможность редактировать и удалять любые публикации.

17. Пользователи с ролью ADMIN должны иметь возможность добавлять университеты с загрузкой логотипа и выбором города, а также дисциплины.

18. Пользователи с ролью ADMIN и MODERATOR должны иметь возможность обновлять логотип университета.

1.5.2 Критический путь пользователей

Критический путь описывает минимальную последовательность действий каждой роли для достижения основной цели.

Гость – найти учебный материал:

1. Открыть главную страницу системы.
2. Перейти в каталог университетов, выбрать нужный университет.
3. Выбрать дисциплину из списка.
4. Открыть публикацию, прочитать текст, просмотреть вложения в подходящем формате.

Студент (USER) – опубликовать материал и сохранить найденный:

1. Зарегистрироваться в системе, затем войти.
2. Перейти к нужной дисциплине, нажать «Добавить публикацию».
3. Заполнить заголовок, текст, прикрепить вложения, при необходимости выбрать личный режим доступа, отправить форму.
4. Открыть публикацию другого пользователя, поставить отметку «Нравится» и добавить её в избранное через кнопку-звёздочку.
5. Открыть боковую панель избранного, убедиться, что публикация сохранена.
6. При необходимости перейти на страницу заявки преподавателя, указать ФИО, загрузить изображение подтверждающего документа и отправить заявку на проверку.
7. Вернуться к своей публикации, изменить текст или режим доступа, затем при необходимости удалить её.

Преподаватель (TEACHER) – опубликовать учебный материал:

1. Войти в систему под учётной записью преподавателя.
2. Создать публикацию по выбранной дисциплине или в разделе «Знания» тем же способом, что и обычный пользователь.
3. Убедиться, что опубликованный материал отображается с пометкой «Материал от преподавателя», а права редактирования и удаления ограничены собственными публикациями.

Модератор (MODERATOR) – проверить заявки и поддержать качество контента:

1. Войти в систему под учётной записью модератора.
2. Открыть раздел заявок преподавателей, просмотреть ФИО заявителя и изображение подтверждающего документа.
3. Одобрить корректную заявку, чтобы система назначила пользователю роль TEACHER.

4. Открыть публикацию, нарушающую правила или не соответствующую назначению платформы, при необходимости отредактировать её содержание или удалить её.

5. При необходимости обновить логотип существующего университета.

Администратор (ADMIN) – назначить модератора и провести модерацию:

1. Войти в систему под учётной записью администратора.
2. Открыть раздел управления пользователями, выбрать нужного пользователя и назначить ему роль MODERATOR.
3. Открыть раздел заявок преподавателей, просмотреть ФИО и изображение подтверждающего документа, затем одобрить корректную заявку.
4. Открыть нежелательную публикацию любого пользователя, при необходимости отредактировать её содержание или удалить её.

1.6 Выводы по главе

В первой главе была рассмотрена предметная область обмена учебными материалами и выявлена основная проблема: учебные ресурсы часто хранятся в разрозненных каналах и не имеют единой структуры. Анализ аналогов показал необходимость разработки специализированной системы с каталогом «университет – дисциплина – публикация», поддержкой вложений, личных материалов, избранного, разграничения пользовательских ролей и проверяемого получения роли преподавателя. На основе этого были сформулированы функциональные требования и критические пути пользователей, которые определяют дальнейшее проектирование приложения ConSuccess.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ

Во второй главе рассматриваются проектные решения и реализация веб-приложения ConSuccess. Первоначально обосновывается выбор набора технологий и приводится описание клиент-серверной архитектуры, отражающее распределение ответственности между клиентской и серверной частями. Далее представлена модель данных в виде реляционной схемы с описанием каждой сущности и связей между ними, что является основой для корректной работы с базой данных на уровне ORM. В разделе проектирования пользовательского интерфейса описаны макеты ключевых страниц системы с обоснованием принятых компоновочных решений и их влияния на удобство навигации. Завершают главу разделы, посвящённые реализации серверной и клиентской частей приложения с описанием модульной структуры, ключевых компонентов и принципов их взаимодействия.

2.1 Выбор набора технологий

Выбор технологий для разработки веб-приложения ConSuccess осуществлялся исходя из требований к системе, необходимости поддержки серверной отрисовки страниц, удобства работы с базой данных и загрузки файлов, а также возможности обеспечить безопасную авторизацию. Набор технологий разделён на серверную и клиентскую части.

2.1.1 Серверная часть

В качестве серверного фреймворка выбран NestJS – фреймворк для Node.js на базе TypeScript. NestJS реализует модульную архитектуру с внедрением зависимостей, предоставляет удобные декораторы для описания маршрутов: @Controller, @Get, @Post, защитников маршрутов: @UseGuards, и перехватчиков файлов: @UseInterceptors [5, 6, 7, 8]. Это позволяет структурировать код по доменным областям: пользователи, уни-

верситеты, дисциплины, публикации, города, избранное, без избыточного дублирования.

Для работы с базой данных применяется Prisma ORM – типобезопасный инструмент доступа к данным, обеспечивающий автоматическую генерацию клиента на основе декларативной схемы `schema.prisma`. Prisma поддерживает миграции, что упрощает развитие структуры базы данных в процессе разработки [9, 10, 11]. В качестве СУБД используется PostgreSQL – реляционная база данных с поддержкой составных первичных ключей, применяемых в таблицах избранного Favorite и отметок Like, уникальных ограничений, внешних ключей и индексов [12, 13, 14].

Аутентификация реализована на основе JWT (JSON Web Token). При успешном входе сервер формирует токен, подписанный секретным ключом `JWT_SECRET`, и возвращает его клиенту [15, 16]. В реальной реализации полезная нагрузка токена содержит поля `id`, `username` и `role`. Клиентская сторона хранит токен в HTTP-only cookie, что исключает доступ к нему из JavaScript и снижает риск XSS-атак. Защищённые маршруты API проверяют токен через AuthGuard, который извлекает Bearer-токен из заголовка `Authorization` и верифицирует его с помощью `JwtService` [17, 18].

Загрузка файлов обеспечивается библиотекой Multer, интегрированной в NestJS через FilesInterceptor [19, 20]. Файлы сохраняются на диск в папку `uploads/` с именем вида `{timestamp}-{random}.{ext}`. Для публикаций принимаются изображения, PDF-документы и MP4-видео; URL сохранённого файла записывается в таблицу `Attachment`. Папка `uploads/` отдаётся клиентам как статические ресурсы по пути `/uploads`.

2.1.2 Клиентская часть

Клиентская часть разработана на основе Next.js (App Router, React 19). Next.js поддерживает серверные компоненты, которые получают данные напрямую на сервере и передают их клиенту в виде готового HTML [21, 22, 23].

Это ускоряет первоначальную загрузку и упрощает работу с cookie на сервере. API-маршруты Next.js из каталога `app/api/` используются как слой-посредник между клиентом и серверной частью NestJS: они читают `cookie access_token`, добавляют его в заголовок `Authorization: Bearer` и перенаправляют запрос на серверную часть [24].

Для стилизации интерфейса применяется Tailwind CSS v4 и собственные переиспользуемые компоненты приложения [25]. Это обеспечивает единый внешний вид всех страниц при минимальном объёме пользовательских стилей. HTTP-запросы к серверной части выполняются через Axios с заранее сконфигурированным экземпляром: в браузере используется публичный адрес из `NEXT_PUBLIC_API_URL`, а на сервере Next.js может применяться внутренний адрес `INTERNAL_API_URL`, что важно при контейнерном запуске [26].

Управление состоянием форм реализовано с помощью React Hook Form: поля проходят клиентскую валидацию, ошибки отображаются под каждым полем, серверные ошибки – через `setError('root')` [27]. После изменения данных серверные маршруты Next.js обновляют связанные страницы через механизм `revalidatePath`, а клиентские компоненты отображают результат операций с помощью всплывающих уведомлений [28].

2.2 Проектирование модели данных

Модель данных системы ConSuccess спроектирована в виде реляционной схемы и описана с помощью Prisma Schema. Схема содержит девять сущностей, связанных отношениями «один ко многим» и «многие ко многим» через таблицы-связки. Такое представление соответствует распространённым подходам к проектированию приложений с доменной моделью и реляционным хранилищем данных [2, 4].

User – пользователь системы. Хранит уникальное имя пользователя `username`, хеш пароля, необязательный путь к аватару, дату создания и роль

USER, TEACHER, MODERATOR или ADMIN. Роль определяет права: обычный пользователь создаёт и редактирует собственные публикации и может отправить заявку на роль преподавателя; преподаватель имеет те же права, но его публикации помечаются в интерфейсе как материалы преподавателя; модератор и администратор могут редактировать и удалять любые публикации, обновлять логотипы университетов и одобрять заявки преподавателей; администратор дополнительно получает интерфейсные возможности управления каталогом.

City – город. Содержит уникальное название. Используется при создании университета для указания местоположения.

University – университет. Содержит название, ссылку на город `cityId` и необязательный путь к логотипу `avatar`. Является корневым уровнем иерархии каталога.

Subject – учебная дисциплина. Привязана к конкретному университету через внешний ключ `universityId`. Образует второй уровень иерархии каталога.

Post – публикация. Содержит заголовок `title`, основной текст `body`, признак видимости `visible`, признак личной публикации `isPrivate`, дату создания, дату обновления, идентификатор автора `userId` и необязательный идентификатор дисциплины `subjectId`. Если `subjectId` равен `null`, публикация относится к разделу «Знания» и не привязана к конкретному университету или дисциплине.

Attachment – вложение. Хранит относительный URL файла на сервере вида `/uploads/имя-файла.расширение`, дату загрузки и идентификатор публикации `postId`. Одна публикация может иметь до пяти вложений: изображений, PDF-документов или MP4-видео.

Favorite – избранное. Таблица-связка между пользователем и публикацией. Имеет составной первичный ключ `[userId, postId]`, что исключает дублирование записей при повторном добавлении одной и той же публикации в избранное. В выборке избранного отображаются публичные публикации и собственные личные публикации текущего пользователя.

Like – отметка «Нравится». Таблица-связка между пользователем и публикацией. Имеет составной первичный ключ [userId, postId], что позволяет одному пользователю поставить только одну отметку на одну публикацию и быстро получать количество отметок через агрегат `_count`.

TeacherApplication – заявка на получение роли преподавателя. Хранит идентификатор пользователя `userId`, ФИО заявителя `fullName`, относительный URL изображения подтверждающего документа `documentUrl`, статус заявки `PENDING` или `APPROVED`, дату создания и данные о рассмотрении. Уникальное ограничение по `userId` не позволяет одному пользователю иметь несколько заявок.

Таблица 2 – Сущности модели данных

Сущность	Ключевые поля	Связи
User	id, username (unique), passwordHash, avatar?, createdAt, role	Post, Favorite, Like, TeacherApplication как заявитель или проверяющий
City	id, name (unique)	University (один ко многим)
University	id, name, cityId, avatar?	City, Subject (один ко многим)
Subject	id, name, universityId	University, Post (один ко многим)
Post	id, title, body, visible, isPrivate, subjectId?, userId, createdAt, updatedAt	Subject (необязательно), User, Attachment, Favorite, Like
Attachment	id, url, postId, uploadedAt	Post
Favorite	userId + postId (составной PK), createdAt	User, Post
Like	userId + postId (составной PK), createdAt	User, Post
TeacherApplication	id, userId (unique), fullName, documentUrl, status, createdAt, reviewedAt?, reviewedById?	User (заявитель), User (проверяющий)

2.3 Проектирование макетов пользовательского интерфейса

На этапе проектирования для ключевых пользовательских сценариев были разработаны каркасные макеты интерфейса. Они отражают структуру страниц, расположение элементов навигации, контентных блоков и интерактивных элементов управления без привязки к итоговой цветовой палитре и фактическому содержанию. В качестве основы принята единая компоновочная схема: горизонтальная навигационная панель в верхней части стра-

ницы, фиксированная боковая панель избранного слева и основная контентная область, занимающая оставшееся пространство. Для каждого выбранного сценария на одном изображении показаны настольная и мобильная версии экрана, что позволяет проверить адаптацию интерфейса до этапа детальной реализации.

2.3.1 Страница каталога вузов

Страница каталога вузов отображает все зарегистрированные в системе учебные заведения. Каркасный макет показывает основную сетку карточек в настольной версии и вертикальный список в мобильной версии. Каждая карточка содержит область для изображения, название вуза, город и ссылку «Открыть». Для авторизованных пользователей с ролью администратора в верхней части страницы доступна кнопка «Добавить», открывающая форму создания нового вуза. Боковая панель избранного в данном разделе отображает последние сохранённые публикации пользователя, обеспечивая быстрый доступ к ним без перехода на другие страницы.

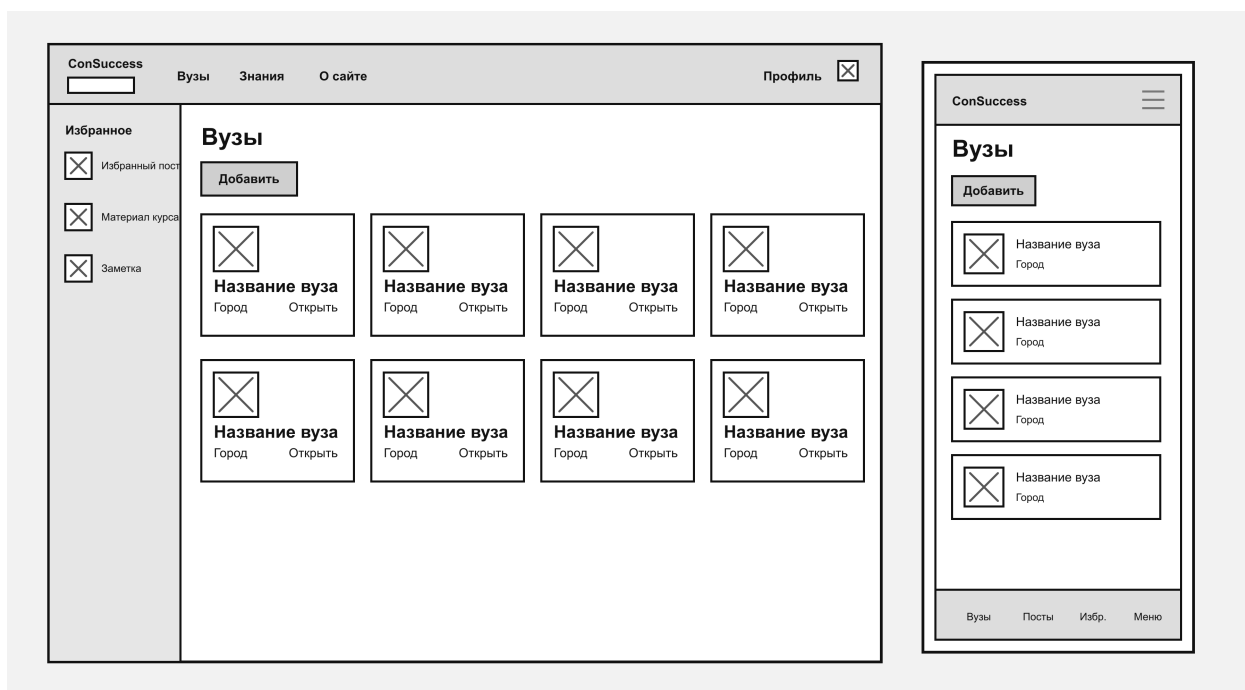


Рисунок 7 – Каркасный макет страницы каталога вузов

2.3.2 Страница публикаций по предмету

Страница публикаций открывается после выбора дисциплины и отображает публичные материалы, привязанные к данному предмету. В заголовке указывается выбранный раздел, а под ним размещается кнопка «Добавить» для перехода к форме создания публикации. Публикации представлены карточками, каждая из которых содержит область вложения, шаблонный заголовок, краткое описание, служебную метку и ссылку «Открыть». Если автор публикации имеет роль преподавателя, на карточке выводится пометка «Преподаватель». В мобильной версии карточки перестраиваются в вертикальный список, сохраняя те же ключевые элементы.

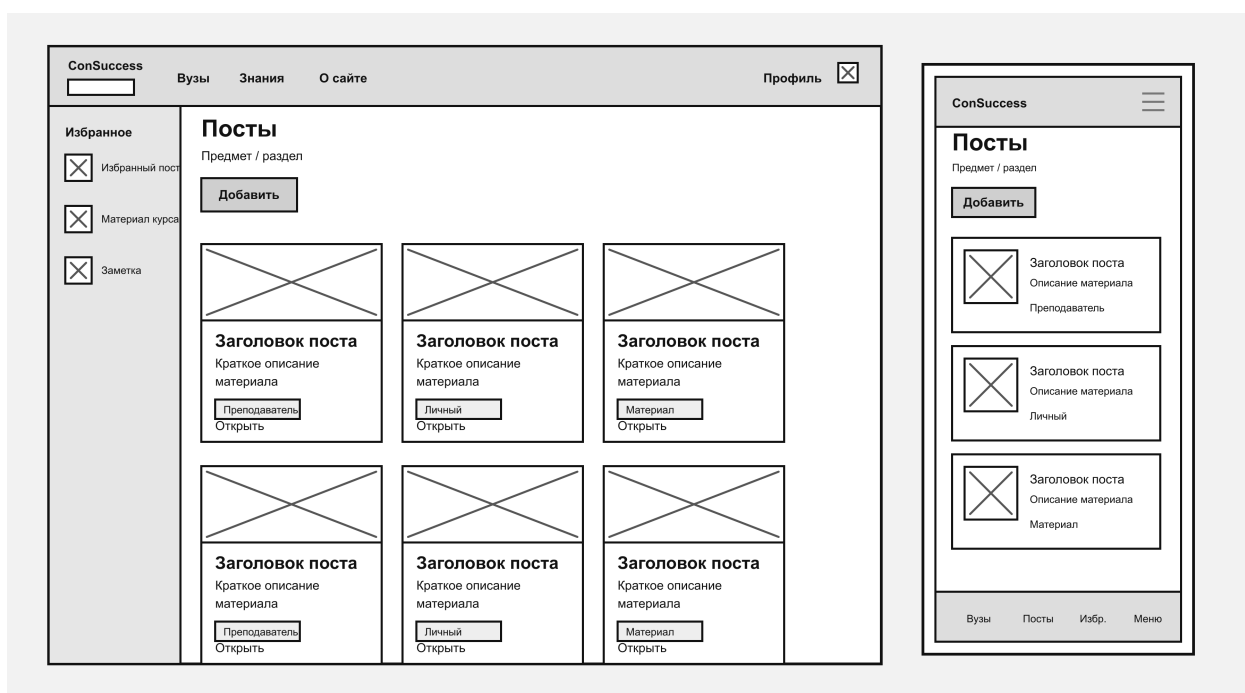


Рисунок 8 – Каркасный макет страницы публикаций по предмету

2.3.3 Страница просмотра публикации

Страница просмотра публикации отображает полное содержимое материала. Внутри основного блока размещаются заголовок публикации, сведения об авторе, действия «Избранное» и «Удалить», а также полный текст материала. Ниже текста располагается область вложений: вместо конкретных изображений на каркасном макете показаны прямоугольные области с диаго-

нальными линиями, обозначающие места для файлов. В мобильной версии те же элементы располагаются в одной колонке, а вложения группируются в компактную сетку.

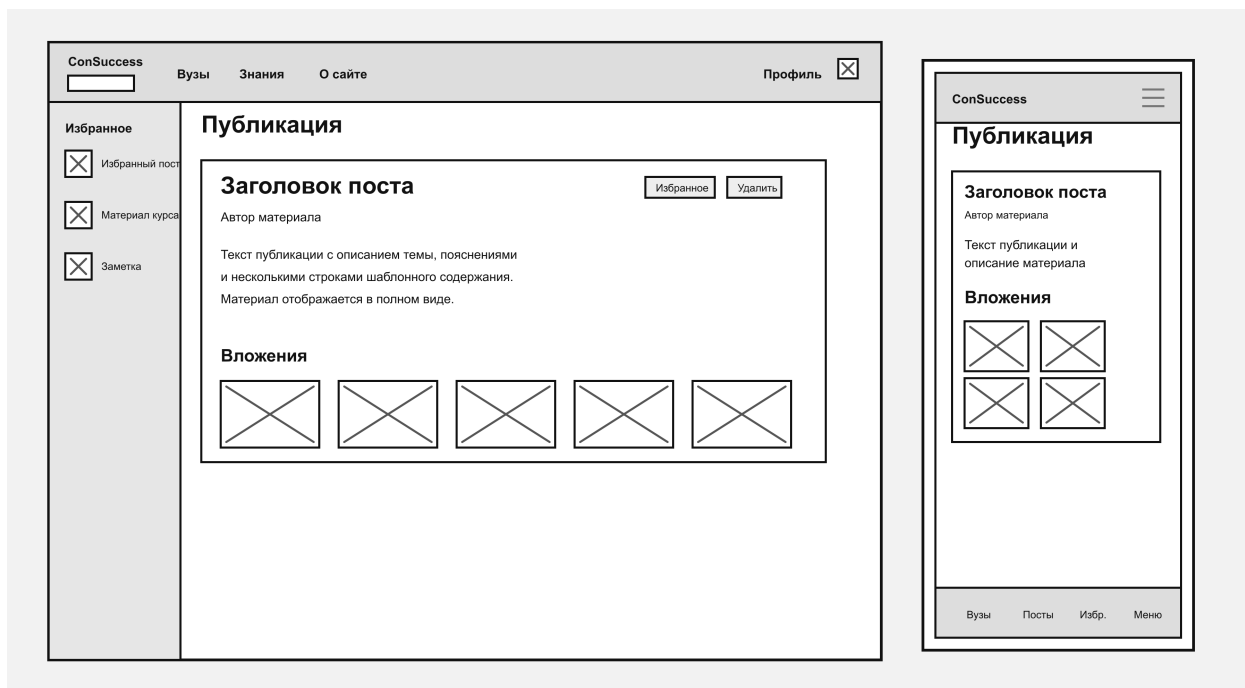


Рисунок 9 – Каркасный макет страницы просмотра публикации

2.3.4 Форма создания публикации

Форма создания публикации используется авторизованным пользователем для добавления нового материала к выбранному предмету. Каркасный макет фиксирует последовательность заполнения формы: поле заголовка, многострочное поле текста, блок загрузки файлов, переключатель личного доступа и кнопку «Опубликовать». В настольной версии форма расположена в центральной области страницы, а в мобильной версии элементы выстроены вертикально и занимают всю доступную ширину экрана. Такой вариант позволяет сохранить единый сценарий создания материала на разных устройствах.

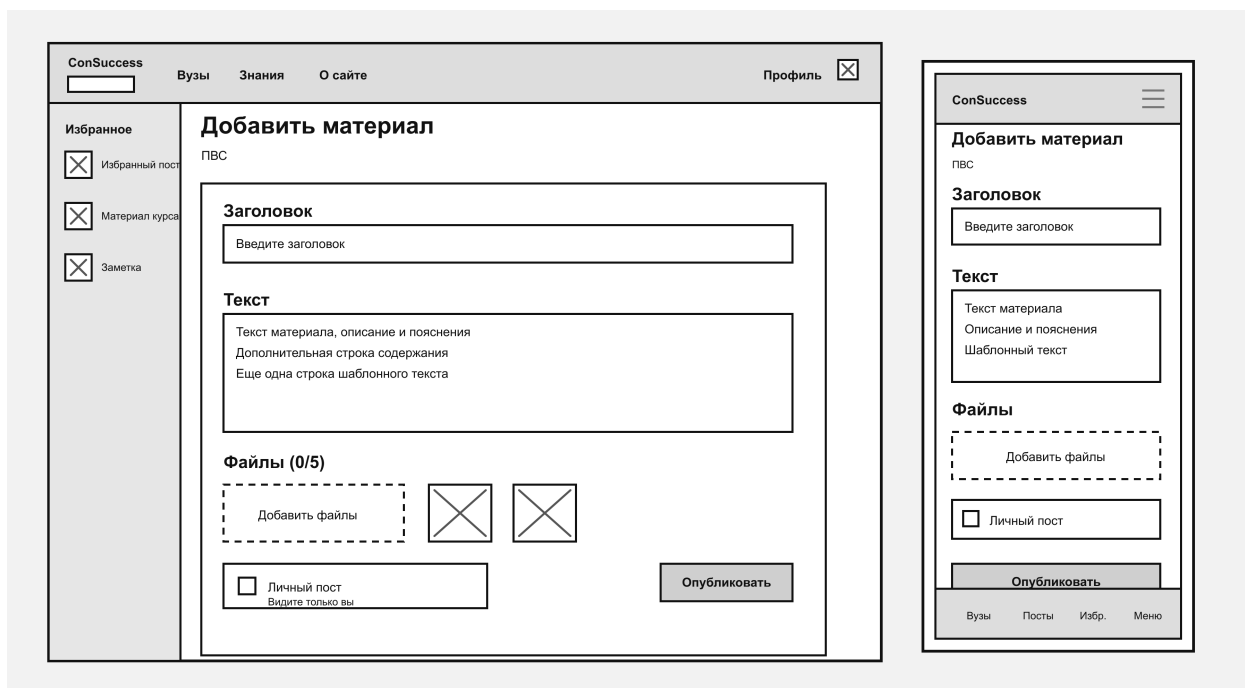


Рисунок 10 – Каркасный макет формы создания публикации

2.3.5 Страница заявки на роль преподавателя и раздел модерации заявок

Страница заявки на роль преподавателя доступна авторизованному пользователю с ролью USER. Пользователь указывает ФИО и загружает одно изображение подтверждающего документа. После отправки форма заменяется блоком состояния заявки: отображаются статус, дата отправки, указанное ФИО и изображение документа. Если заявка уже одобрена или пользователь уже имеет роль преподавателя, страница сообщает о текущем состоянии и не предлагает повторную отправку.

Для пользователей с ролями ADMIN и MODERATOR предусмотрен отдельный раздел заявок. В нём отображается список заявок с данными пользователя, ФИО, датой создания, изображением документа и кнопкой одобрения. При одобрении система изменяет статус заявки и назначает пользователю роль TEACHER. Такой подход отделяет пользовательский сценарий подачи заявки от административного сценария проверки и сохраняет контроль за выдачей признака преподавателя.

2.4 Реализация серверной части

Серверная часть веб-приложения ConSuccess реализована на фреймворке NestJS с использованием TypeScript. Серверная часть предоставляет набор маршрутов REST API, к которым клиент обращается для получения и изменения данных каталога, работы с публикациями, авторизацией, отметками «Нравится» и загрузкой файлов.

2.4.1 Структура серверного приложения

Серверное приложение построено вокруг корневого модуля AppModule, в котором регистрируются все контроллеры и сервисы. В рамках проекта принята плоская организация: вместо создания отдельного модуля для каждой доменной области все контроллеры и сервисы подключаются непосредственно в корневом модуле. Это оправдано относительно небольшим количеством доменных областей и упрощает навигацию по кодовой базе.

Корневой модуль импортирует ConfigModule для чтения переменных окружения, PrismaModule как глобальный модуль, предоставляющий PrismaService, и AuthModule, инкапсулирующий AuthService, AuthController и JwtModule с настройкой секрета из ConfigService.

Точка входа приложения main.ts создаёт экземпляр NestExpressApplication, настраивает CORS для источников из переменной окружения FRONTEND_ORIGIN, создаёт каталог uploads/ при его отсутствии и регистрирует его как источник статических ресурсов по пути /uploads. Сервер запускается на адресе 0.0.0.0, что позволяет использовать его как локально, так и внутри Docker-сети. Это позволяет клиенту запрашивать загруженные файлы напрямую у серверной части без дополнительного перенаправления через промежуточный маршрут.

2.4.2 Доменные сервисы и контроллеры

Для каждой сущности каталога в проекте реализованы пары «контроллер – сервис». Контроллеры отвечают за приём HTTP-запросов, извлечение параметров маршрута и тела запроса, применение защитников AuthGuard и перехватчиков файлов. Сервисы инкапсулируют бизнес-логику и работу с базой данных через PrismaService.

1. UsersController и UsersService управляют пользователями: создание, получение по идентификатору, получение по имени пользователя. Пароли пользователей хранятся в виде bcrypt-хеша с десятью раундами соления; функции хеширования вынесены в модуль utils/crypt.ts.

2. UniversitiesController и UniversitiesService предоставляют операции получения списка университетов и отдельного университета по идентификатору, создание нового университета с загрузкой логотипа и указанием города, а также обновление логотипа существующего университета пользователями с ролями ADMIN и MODERATOR. При создании и обновлении логотипа контроллер использует FileInterceptor.

3. CitiesController и CitiesService отвечают за получение и создание городов, используемых при регистрации новых университетов.

4. SubjectsController и SubjectsService реализуют операции для дисциплин в контексте выбранного университета. Все операции принимают universityId как параметр маршрута, что обеспечивает иерархическую изоляцию дисциплин по университетам.

5. PostsController и PostsService управляют публикациями в рамках выбранной дисциплины, а также публикациями раздела «Знания» через отдельный KnowledgePostsController. Маршрут создания публикации защищён AuthGuard, использует FilesInterceptor для приёма до пяти вложений, применяет лимиты публикации и сохраняет файлы в каталог uploads/. Для личных публикаций дополнительно проверяется автор или привилегированная роль.

6. `FavoritesController` и `FavoritesService` реализуют добавление и удаление публикаций из избранного для текущего авторизованного пользователя, а также получение списка избранных публикаций. Идентификатор пользователя извлекается из JWT-токена, а недоступные личные публикации исключаются из выдачи.

7. `LikesController` и `LikesService` реализуют добавление и удаление отметки «Нравится», а также получение списка отмеченных публикаций текущего пользователя. Повторное добавление не создаёт дубль благодаря составному первичному ключу.

8. `TeacherApplicationsController` и `TeacherApplicationsService` реализуют отправку заявки на роль преподавателя, получение текущей заявки пользователя, просмотр всех заявок пользователями с ролями `ADMIN` и `MODERATOR`, а также одобрение заявки с назначением роли `TEACHER`. Для изображения подтверждающего документа используется `FileInterceptor`.

2.4.3 Модуль авторизации

Модуль авторизации инкапсулирует логику входа пользователя, регистрации и проверку доступа к защищённым маршрутам API. `AuthService` принимает имя пользователя и пароль, запрашивает пользователя через `UserService.getUserByName`, сравнивает переданный пароль с сохранённым хешем через `bcrypt`. В случае успеха формируется JWT-токен с полезной нагрузкой `{ id, username, role }` и подписью секретом `JWT_SECRET`. Токен возвращается вызывающему серверному маршруту `Next.js`, после чего сохраняется в HTTP-only cookie и не передаётся браузерному JavaScript-коду.

`AuthController` предоставляет маршруты `POST /auth/login`, `POST /auth/register` и защищённый `GET /auth/me`. Последний возвращает данные текущего пользователя, что позволяет клиенту проверить валидность сессии и получить актуальные сведения о роли.

AuthGuard реализует интерфейс CanActivate. Guard извлекает Bearer-токен из заголовка Authorization, проверяет его подпись через JwtService.verifyAsync и при успехе прикрепляет раскодированную полезную нагрузку к объекту запроса request.user. При отсутствии или недействительности токена guard возбуждает UnauthorizedException, что приводит к ответу клиенту со статусом 401.

2.4.4 Работа с базой данных через Prisma

Для доступа к базе данных используется Prisma ORM. Декларативная схема базы данных описана в файле schema.prisma и содержит модели User, City, University, Subject, Post, Attachment, Favorite, Like, TeacherApplication. Для перечислимых значений роли пользователя определён enum Role со значениями USER, TEACHER, MODERATOR и ADMIN, а для заявок преподавателей – enum TeacherApplicationStatus со значениями PENDING и APPROVED.

Все изменения структуры базы данных выполняются через миграции Prisma. Миграции хранятся в каталоге prisma/migrations/ под именами, отражающими дату и семантику изменения: изменение роли, добавление роли преподавателя, добавление поля аватара, связь с городом, добавление поля видимости публикации, добавление отметок Like, добавление признака личной публикации и добавление заявок на роль преподавателя. Такой подход обеспечивает воспроизводимость среды и упрощает развёртывание на новых серверах.

PrismaService расширяет PrismaClient и управляет подключением к базе данных в рамках жизненного цикла NestJS. Сервис зарегистрирован в глобальном модуле PrismaModule, что делает его доступным для всех сервисов без повторного импорта.

Запросы к базе данных формируются через типобезопасный Prisma Client. Например, при получении публикации вместе с вложениями исполь-

зуется включение связанной сущности `include: { attachments: true }`, для отображения количества отметок применяется `_count: { select: { likes: true } }`, а при фильтрации избранного и отметок используется составной `where` с идентификатором пользователя.

2.4.5 Загрузка и хранение вложений

Для загрузки вложений используется библиотека `Multer`, интегрированная в `NestJS` через декораторы `FilesInterceptor` и `FileInterceptor`. Приёмник файлов для публикаций настраивается через объект `postAttachmentUploadOptions`. Для сохранения файлов используется `diskStorage`; имя файла генерируется по шаблону `{timestamp}-{random}.{ext}`, что исключает коллизии имён при одновременных загрузках. Проверка принимает изображения, PDF-документы и MP4-видео; для PDF и видео дополнительно проверяются MIME-тип и расширение. Для логотипов, аватаров и подтверждающих документов в заявке преподавателя используется вариант загрузки только изображений.

После успешного сохранения файла `multer` добавляет его метаданные в массив `files` запроса. Сервис публикаций перебирает массив и создаёт запись `Attachment` для каждого файла: в поле `url` записывается относительный путь `/uploads/имя-файла.расширение`, который клиент может дополнять базовым URL сервера для получения файла через статический маршрут.

2.4.6 Обработка ошибок и валидация

Для обработки ошибок используются встроенные исключения `NestJS`: `UnauthorizedException` (401), `ForbiddenException` (403), `NotFoundException` (404), `BadRequestException` (400). Исключения автоматически преобразуются в HTTP-ответы с соответствующими статусами и сообщениями.

Валидация входных данных выполняется на уровне контроллеров и сервисов. При отсутствии обязательных полей или неверном формате данных возбуждается `BadRequestException`. При создании публикации сервис также проверяет количественные ограничения: не более десяти публикаций за текущий день и не более пятидесяти публикаций всего для одного пользователя. Для защиты от SQL-инъекций используется параметризация запросов Prisma: все значения передаются в виде аргументов, а не конкатенируются в строку запроса, что исключает возможность внедрения постороннего SQL-кода.

Таким образом, серверная часть приложения обеспечивает модульную структуру, безопасную авторизацию через JWT, надёжную работу с базой данных через Prisma, корректную обработку загружаемых вложений, управление доступом к личным публикациям, обработку заявок на роль преподавателя и предсказуемую обработку ошибок. Данные архитектурные и реализационные решения формируют устойчивую основу для расширения функциональности и поддержки приложения в дальнейшем.

2.5 Реализация клиентской части

Клиентская часть веб-приложения ConSuccess реализована на фреймворке Next.js с моделью App Router и использованием React 19. Next.js сочетает преимущества серверной и клиентской отрисовки: серверные компоненты получают данные непосредственно на сервере и отправляют клиенту уже сформированный HTML, а клиентские компоненты обеспечивают интерактивность: формы, галереи, работу с состоянием. Такой подход снижает первоначальное время загрузки страниц и упрощает управление секретами авторизации, поскольку JWT-токен не передаётся в JavaScript-код браузера.

2.5.1 Структура клиентского приложения

Клиентское приложение построено в соответствии с соглашениями App Router. Основные каталоги:

1. `app/` – корневой каталог маршрутизации. Содержит разметку приложения `layout.tsx`, глобальные стили и метаданные.

2. `app/(user)/` – группа маршрутов пользовательской зоны, оформленная как скобочная группа Next.js. Общий макет содержит горизонтальную навигационную шапку, а также компонент `FavoritesSidebar`. В этой зоне размещена страница `/teacher-application` для отправки и просмотра состояния заявки преподавателя.

3. `app/(admin)/` – группа маршрутов административной и модераторской зоны. Она содержит разделы управления пользователями, модерации публикаций и проверки заявок на роль преподавателя.

4. `app/api/` – серверные обработчики маршрутов `route.ts`, играющие роль посредника между клиентом и серверной частью. Вся логика работы с `cookie access_token` вынесена сюда.

5. `app/_components/` – переиспользуемые компоненты интерфейса: `Header`, `Navlink`, `HeaderProfile`, `UserActions`, `PostCard`, `KnowledgePostCard`, `FavoriteButton`, `LikeButton`, `LikeCount`, `AttachmentTile`, `ImageGallery`, `TeacherMaterialBadge`.

6. `app/_icons/` – централизованный реестр иконок. SVG-файлы импортируются из каталога `assets/` и реэкспортируются по именам.

7. `shared/api/` – клиентский слой работы с API: базовый экземпляр `Axios`, строковые константы маршрутов, функции доступа к конкретным ресурсам.

8. `shared/types/` – TypeScript-типы сущностей домена: `City`, `University`, `Subject`, `Post`, `Attachment`, `Favorite`, `Like`, `User`, `TeacherApplication`.

9. `shared/lib/` – вспомогательные функции клиентской части, включая определение типа вложения и выбор файла для предпросмотра карточки.

Помимо основной зоны каталога, в приложении реализованы страницы `/login` и `/register`, отдельный раздел `/knowledge` для публикаций, не привязанных к конкретной дисциплине, со страницами добавления и просмотра публикаций, маршрут `/universities/[id]/subjects/[subjectId]/private` для личных материалов пользователя по выбранной дисциплине, а также маршруты `/teacher-application` и `/moderator/teacher-applications` для подачи и рассмотрения заявок преподавателей.

2.5.2 Страницы и серверные компоненты

Большинство страниц приложения являются серверными компонентами. Такие компоненты имеют доступ к `cookies()` и могут выполнять запросы к API напрямую на сервере. Это позволяет выполнять условное отображение на основе признака авторизации: например, кнопки «Добавить» на страницах каталога университетов и дисциплин отображаются только пользователям с ролью администратора, кнопка «Мои личные» со счётчиком личных публикаций – авторизованным пользователям, карточка перехода к заявке преподавателя – на информационной странице, а кнопки редактирования и удаления публикации – только автору публикации или модератору. Правила владения публикацией проверяются по совпадению `userId` текущего пользователя с автором публикации.

Для страниц, требующих интерактивности: формы публикации, галерея с полноэкранным просмотром, переключатель избранного, кнопка отметки «Нравится», переключатель режима доступа и обновление логотипа университета, создаются отдельные клиентские компоненты в каталогах `_components/` рядом с соответствующей страницей. Сервер формирует необходимые данные и передаёт их в виде свойств клиентскому компоненту.

Разделение ответственности между серверным и клиентским компонентом улучшает производительность и обеспечивает корректную работу состояния формы.

Страницы с параметрами маршрута `[id]`, `[subjectId]`, `[postId]` используют асинхронные функции серверных компонентов и возвращают разметку на основе полученных данных. При отсутствии запрашиваемой сущности вызывается `notFound()`, что приводит к отображению страницы 404. Несколько параллельных запросов данных на одной странице выполняются через `Promise.all`, что уменьшает общее время ожидания [22].

2.5.3 Серверные маршруты в роли посредника

Next.js API-маршруты `app/api/.../route.ts` используются для передачи запросов от клиента к серверной части NestJS. Данный подход решает несколько задач: централизует обращения к защищённым маршрутам серверной части, управляет `cookie access_token`, поддерживает HTTP-only cookie и снижает риск XSS-атак.

Все маршруты-посредники построены по единой схеме. Сначала читается `access_token` из cookie; при отсутствии возвращается статус 401. Затем формируется запрос через экземпляр `axiosApi` к соответствующему маршруту серверной части с заголовком `Authorization: Bearer`. Для JSON-запросов тело читается через `await req.json()`; так работают маршруты изменения режима доступа и отметок «Нравится». Для запросов с загрузкой файлов тело передаётся в виде `Buffer.from(await req.arrayBuffer())` вместе с исходным заголовком `Content-Type`, что сохраняет границу `multipart` и обеспечивает корректную обработку файлов на серверной части. При ошибке `Axios` статус и тело ответа серверной части передаются клиенту без изменений.

Отдельно стоит отметить маршрут авторизации `POST /api/auth/login`: он передаёт запрос на серверную часть, а при успешном

ответе сохраняет полученный JWT-токен в HTTP-only cookie. Этот механизм полностью инкапсулирует работу с токеном – клиентский код не видит и не сохраняет токен самостоятельно.

2.5.4 Формы и работа с данными

Формы регистрации, входа, создания университета, создания дисциплины, создания и редактирования публикации, а также форма заявки на роль преподавателя реализованы с использованием библиотеки React Hook Form. Она предоставляет удобный API для регистрации полей, отслеживания состояния валидации и отправки формы. В формах публикации используется единый набор ограничений для вложений: не более пяти файлов, разрешены `image/*`, `video/mp4` и `application/pdf`. Для тематических публикаций добавлен флажок личного режима доступа. Форма заявки преподавателя содержит поле ФИО и загрузку одного изображения подтверждающего документа.

Ключевые соглашения форм: ошибки валидации полей отображаются непосредственно под соответствующим полем; ошибки сервера сохраняются через `setError('root { message })` и отображаются над полями формы; состояние отправки отслеживается через `formState.isSubmitting` и используется для блокировки кнопки отправки; над заголовком формы размещается ссылка «Назад» для быстрого возврата к предыдущей странице.

После изменения данных серверные маршруты Next.js вызывают `revalidatePath`, чтобы обновить связанные страницы и не показывать пользователю устаревшие данные [28]. Всплывающие уведомления об успехе или ошибке операций реализованы через библиотеку Sonner, что обеспечивает единообразный и неблокирующий пользовательский опыт.

2.5.5 Отображение публикаций и загрузка вложений

Публикации отображаются в виде карточек: `PostCard` для публикаций дисциплины и `KnowledgePostCard` для раздела «Знания». Каждая карточ-

ка содержит предпросмотр первого вложения, заголовок, краткий фрагмент текста, счётчик отметок «Нравится» и ссылку «Открыть» на полную страницу публикации. Для материалов преподавателя используется компонент `TeacherMaterialBadge`. Компонент `AttachmentTile` определяет тип вложения: изображение отображается как миниатюра, видео – как область предпросмотра с пиктограммой воспроизведения, PDF-документ – как плитка с обозначением файла. При отображении вложений соблюдаются следующие правила:

1. Файлы, отдаваемые серверной частью по пути `/uploads/...`, префиксуются базовым URL сервера `apiURL` из модуля `shared/api/config`.

2. Компонент `next/image` используется с параметром `unoptimized: true`, поскольку оптимизатор Next.js по умолчанию может отклонять запросы к приватным IP-адресам [29].

3. Для отображения полного изображения без обрезки используется `object-contain` вместо `object-cover`. Это важно для учебных материалов, где часть содержимого не должна обрезаться.

4. PDF-документы открываются в новой вкладке, а видео в формате MP4 воспроизводится в полноэкранном просмотре с элементами управления.

5. В файле `next.config.ts` в секции `images.remotePatterns` разрешены хосты `localhost:5000` и `127.0.0.1:5000`, используемые при локальной разработке.

Для просмотра изображения или видео в полноэкранном режиме реализован клиентский компонент `ImageGallery`. Компонент управляет состоянием открытой карточки, блокирует прокрутку страницы при открытом вложении и закрывает полноэкранный просмотр по клику вне содержимого или по нажатию клавиши `Escape`.

Переключатель избранного реализован как отдельный клиентский компонент `FavoriteButton` в форме звёздочки. При нажатии он отправляет запрос на добавление или удаление публикации из избранного и обновляет соответствующее состояние. В избранном отображаются публичные публика-

ции и собственные личные публикации пользователя; личные материалы помечаются отдельной меткой. Отметка «Нравится» реализована компонентом `LikeButton`: он меняет состояние на клиенте, отправляет запрос на серверный маршрут и обновляет счётчик. Список избранного формируется в боковой панели `FavoritesSidebar`, доступной на всех страницах пользовательской зоны.

2.5.6 Стилизация интерфейса

Для стилизации используется Tailwind CSS версии 4. Базовые элементы интерфейса, включая кнопки, поля ввода, карточки и уведомления, реализованы как собственные компоненты приложения с использованием утилитарных классов. Переиспользуемые элементы размещаются в каталоге `app/_components/`, а компоненты отдельных страниц – в локальных каталогах `_components`. Такой подход обеспечивает единообразный визуальный стиль и минимизирует количество пользовательских стилей. Иконки берутся из библиотеки Lucide. При построении компонентов учитываются общие принципы повторного использования и композиции интерфейсных элементов [30].

Цветовая палитра клиентской части строится вокруг основного синего цвета и нейтральных оттенков для фона, текста и границ. Явно заданные цвета хранятся в глобальных стилях приложения, а дополнительные состояния берутся из стандартной палитры Tailwind CSS версии 4. Для таких цветов ниже приведены коды в формате OKLCH, используемом библиотекой.

1. `#067BC2` — синий, основной цвет приложения; используется для главных кнопок, активных элементов навигации, ссылок, выделения выбранных состояний и фона всплывающих уведомлений об успешных действиях.

2. `#0569A8` — темно-синий, вспомогательный контрастный оттенок; применяется для рамки всплывающего уведомления, чтобы отделить его от фона страницы.

3. #F7F9FC — очень светлый серо-голубой; используется как общий фон страниц, чтобы белые карточки, формы и области ввода визуально отделились от рабочей области.

4. #FFFFFF — белый; используется для карточек, модальных окон, полей форм, выпадающих областей и текста на насыщенном синем фоне.

5. `oklch(20.8% 0.042 265.755)` — темный графитовый оттенок `slate-900`; применяется для основных заголовков и наиболее значимого текста.

6. `oklch(55.4% 0.046 257.417)` и `oklch(55.6% 0 0)` — серые оттенки `slate-500` и `neutral-500`; используются для второстепенного текста, описаний, подписей и менее приоритетных ссылок.

7. `oklch(92.2% 0 0)` и `oklch(92.8% 0.006 264.531)` — светло-серые оттенки `neutral-200` и `gray-200`; применяются для рамок, разделителей, контуров полей ввода и элементов загрузки.

8. `oklch(97% 0.014 254.604)` и `oklch(54.6% 0.245 262.881)` — светло-синий и насыщенный синий оттенки `blue-50` и `blue-600`; используются в информационных блоках, значках дисциплин и состояниях выбранных элементов.

9. `oklch(63.7% 0.237 25.331)` и `oklch(57.7% 0.245 27.325)` — красные оттенки `red-500` и `red-600`; применяются для ошибок, предупреждений, кнопок удаления и отрицательных состояний.

10. `oklch(75% 0.183 55.934)` и `oklch(76.9% 0.188 70.08)` — оранжевый и янтарный оттенки `orange-400` и `amber-500`; используются для визуального выделения избранных материалов, значков со звездой и отдельных управляющих элементов.

11. `rgba(0, 0, 0, 0.4)` и `rgba(0, 0, 0, 0.8)` — черный цвет с прозрачностью; применяется для затемнения фона модальных окон и контрастных подсказок.

Компоновка страниц приложения использует flex-контейнеры: боковая панель слева и основная контентная область с классами `flex-1 min-w-0`.

Модификатор `min-w-0` необходим для корректной работы переноса длинных строк внутри контентной области. Для переноса длинных непрерывных строк в Tailwind CSS v4 используется утилита `wrap-break-word`.

Таким образом, клиентская часть приложения обеспечивает удобную иерархическую навигацию по каталогу, безопасную работу с токенами через серверные API-маршруты, качественное отображение вложений с поддержкой полноэкранного просмотра, пользовательский сценарий подачи заявки преподавателя, раздел проверки заявок для привилегированных ролей, а также единообразный и адаптивный пользовательский интерфейс. Разделение ответственности между серверными и клиентскими компонентами позволяет использовать сильные стороны обоих подходов и обеспечивает высокую производительность приложения.

2.6 Контейнеризация и локальное развёртывание

Для воспроизводимого запуска приложения подготовлена контейнерная конфигурация Docker. В корне проекта находится файл `docker-compose.yml`, описывающий три сервиса:

1. `db` – контейнер PostgreSQL 17 с постоянным томом `postgres_data` для хранения данных и проверкой готовности через `pg_isready`;
2. `backend` – контейнер серверной части NestJS, собираемый из каталога `Backend`; при запуске он применяет миграции Prisma, открывает порт 5000, использует переменную `DATABASE_URL` для подключения к базе данных внутри Docker-сети и сохраняет загруженные файлы в том `backend_uploads`;
3. `frontend` – контейнер клиентской части Next.js, собираемый из каталога `Frontend`; он открывает порт 3000, использует `NEXT_PUBLIC_API_URL` для запросов из браузера и `INTERNAL_API_URL` для серверных запросов внутри Docker-сети.

Для серверной и клиентской частей созданы отдельные файлы `Dockerfile`. Серверный образ основан на `node:22-bookworm-slim`, устанавливает зависимости через `npm ci`, генерирует `Prisma Client`, собирает `NestJS`-приложение и запускает производственную сборку. Клиентский образ также использует `Node.js 22`, устанавливает зависимости, выполняет сборку `Next.js` и запускает приложение командой `npm run start`.

В конфигурации `docker-compose.yml` заданы зависимости между сервисами: серверная часть запускается после готовности базы данных, а клиентская часть – после готовности серверной части. Для серверного контейнера определена проверка доступности HTTP-маршрута, что позволяет Docker дожидаться фактической готовности API. Такое разделение упрощает локальное развёртывание проекта, делает окружение одинаковым на разных компьютерах и отделяет данные базы и загруженные файлы от жизненного цикла контейнеров.

2.7 Выводы по главе

Во второй главе были спроектированы и реализованы основные части веб-приложения `ConSuccess`: модель данных, серверная часть на `NestJS`, клиентская часть на `Next.js`, механизмы авторизации, публикаций, вложений, избранного, заявок на роль преподавателя и контейнерного запуска. Принятые архитектурные решения обеспечивают разделение ответственности между клиентом, сервером и базой данных, а также позволяют расширять систему без переработки базовой структуры. Реализация соответствует требованиям, сформулированным в первой главе, и подготавливает основу для проверки качества системы.

3 ТЕСТИРОВАНИЕ И ОЦЕНКА СИСТЕМЫ

Третья глава посвящена вопросам оценки качества разработанной системы. В ней рассматриваются сценарии функционального тестирования для пользовательских ролей: гость, студент, преподаватель, модератор и администратор, меры по обеспечению информационной безопасности и возможности масштабирования и дальнейшего развития платформы ConSuccess. Целью раздела является подтверждение соответствия системы функциональным требованиям, сформулированным в первой главе, анализ потенциальных угроз и описание перспективных направлений развития [31].

3.1 Сценарии функционального тестирования

Сценарий функционального тестирования составлен на основе критического пути пользователей и охватывает ключевые роли: гость, студент (USER), преподаватель (TEACHER), модератор (MODERATOR) и администратор (ADMIN). Для каждой роли определены тестируемые шаги, ожидаемый результат и критерий прохождения. Автоматизация таких проверок в дальнейшем может быть выполнена средствами Vitest, React Testing Library и Jest [32, 33, 34].

3.1.1 Сценарий тестирования для роли «Гость» – структурная навигация к учебному материалу

Цель: убедиться, что неавторизованный пользователь может просматривать каталог и публикации без входа в систему.

Таблица 3 – Сценарий тестирования роли «Гость»

№	Шаг	Ожидаемый результат	Статус
1	Открыть главную страницу системы без авторизации.	Отображается список последних публикаций. Кнопка «Логин» видна в навигации.	
2	Перейти в раздел «Вузы».	Отображается сетка карточек университетов с логотипами, названиями и городами.	
3	Выбрать университет из списка.	Открывается страница дисциплин выбранного вуза. Присутствует ссылка «Назад».	
4	Выбрать дисциплину из списка.	Отображаются карточки публичных публикаций с предпросмотром вложений, фрагментом текста и счётчиком отметок «Нравится».	
5	Открыть публикацию по ссылке «Открыть».	Отображается полный текст публикации, данные об авторе, дата, вложения и количество отметок «Нравится». Кнопки «Редактировать» и «Удалить» не отображаются.	
6	Открыть вложение на странице публикации.	Изображение или видео открывается в полноэкранном режиме; PDF-документ открывается отдельной ссылкой.	

3.1.2 Сценарий тестирования для ролей «Студент (USER)» и «Преподаватель (TEACHER)» – публикация материала и работа с избранным

Цель: убедиться, что авторизованный пользователь может создавать, редактировать и удалять публикации, управлять личными материалами, ставить отметки «Нравится» и пользоваться механизмом избранного, а публикации преподавателя корректно выделяются в интерфейсе.

Таблица 4 – Сценарий тестирования ролей «Студент (USER)» и «Преподаватель (TEACHER)»

№	Шаг	Ожидаемый результат	Статус
1	Зарегистрироваться в системе с уникальным именем пользователя и паролем.	Учётная запись создана. Выполнен автоматический вход или предложена страница входа.	
2	Войти в систему.	В навигации отображается имя пользователя и кнопка выхода. Боковая панель «Избранное» доступна.	
3	Перейти к нужной дисциплине и нажать «Добавить».	Открывается форма создания публикации с полями «Заголовок», «Текст», «Файлы» и флажком личного режима.	
4	Заполнить заголовок и текст, прикрепить разрешённые вложения, нажать «Добавить».	Публичная публикация создана. Выполнено перенаправление на страницу дисциплины. Новая публикация отображается в списке.	
5	Создать публикацию с включённым флажком «Личный пост».	Публикация создана, но не отображается в общем списке дисциплины.	
6	Нажать «Мои личные» на странице дисциплины.	Открывается список личных материалов текущего пользователя по выбранной дисциплине; на кнопке отображается количество таких материалов.	
7	Открыть доступную публикацию, поставить отметку «Нравится» и нажать кнопку-звёздочку «В избранное».	Счётчик отметок увеличивается, публикация добавлена в избранное, в боковой панели появляется ссылка на сохранённую публикацию; собственная личная публикация также отображается в избранном с меткой «Личный».	
8	Повторно нажать кнопку-звёздочку и кнопку отметки «Нравится» на той же публикации.	Публикация удалена из избранного, отметка убрана, соответствующие счётчики и боковая панель обновлены.	
9	Открыть собственную публикацию и нажать «Редактировать» или кнопку изменения режима доступа.	Открывается форма редактирования с текущими данными; после сохранения изменения отображаются на странице публикации, а режим доступа обновляется.	
10	Открыть собственную публикацию и нажать «Удалить».	Публикация скрыта из пользовательского каталога. Выполнено перенаправление на страницу дисциплины. Публикация отсутствует в списке.	
11	Открыть страницу заявки преподавателя, указать ФИО и загрузить изображение подтверждающего документа.	Заявка создана, пользователь видит её статус, ФИО и загруженное изображение.	
12	Войти под учётной записью с ролью TEACHER и создать публикацию.	Публикация отображается с меткой «Материал от преподавателя», при этом права совпадают с правами обычного пользователя.	

3.1.3 Сценарий тестирования для ролей «Администратор (ADMIN)» и «Модератор (MODERATOR)» – управление каталогом и модерация

Цель: убедиться, что администратор может добавлять университеты и дисциплины, администратор и модератор могут обновлять логотипы университетов, редактировать и удалять публикации любых пользователей и рассматривать заявки на получение роли преподавателя.

Таблица 5 – Сценарий тестирования ролей «Администратор (ADMIN)» и «Модератор (MODERATOR)»

№	Шаг	Ожидаемый результат	Статус
1	Войти в систему под учётной записью с ролью ADMIN.	Выполнен вход. На страницах каталога отображается кнопка «Добавить» для университетов и дисциплин.	
2	Перейти в раздел «Вузы» и нажать «Добавить». Заполнить название, выбрать город, загрузить логотип.	Университет создан и отображается в каталоге с загруженным логотипом и городом.	
3	На карточке существующего университета выбрать новый файл логотипа.	Логотип обновлён, карточка университета отображает новое изображение.	
4	Перейти на страницу созданного университета и нажать «Добавить» для создания дисциплины.	Дисциплина создана и отображается в списке предметов университета.	
5	Открыть раздел заявок преподавателей.	Отображается список заявок с ФИО заявителей, статусами и изображениями подтверждающих документов.	
6	Нажать «Одобрить» для ожидающей заявки.	Заявка получает статус «Одобрена», а пользователь получает роль TEACHER.	
7	Открыть публикацию любого пользователя.	На странице публикации отображаются действия модерации вне зависимости от авторства: «Редактировать» и «Удалить».	
8	Нажать «Редактировать» на публикации другого пользователя, изменить текст и сохранить форму.	Изменения сохранены и отображаются на странице публикации.	
9	Нажать «Удалить» на публикации другого пользователя.	Публикация скрыта. Выполнено перенаправление на страницу дисциплины. Публикация отсутствует в списке.	

3.2 Информационная безопасность

Вопросы информационной безопасности имеют ключевое значение для веб-приложений, обрабатывающих пользовательские учётные записи и загружаемый контент. В системе ConSuccess реализован набор мер, направленных на защиту учётных данных пользователей, предотвращение несанкционированного доступа к защищённым операциям и противодействие распространённым уязвимостям веб-приложений [35, 36, 37, 38]. В данном разделе описаны принятые проектные решения и особенности их реализации.

3.2.1 Хранение паролей

Пароли пользователей никогда не сохраняются в базе данных в открытом виде. При регистрации пользователя пароль хешируется алгоритмом bcrypt с коэффициентом десяти раундов соления. Функции хеширования и сравнения вынесены в модуль `utils/crypt.ts` серверной части. При входе в систему введённый пароль сравнивается с сохранённым хешем через функцию `bcrypt`, что исключает возможность восстановления исходного пароля даже при утечке базы данных. Выбор `bcrypt` обусловлен его устойчивостью к перебору: алгоритм намеренно спроектирован медленным и адаптивным, что замедляет атаки перебором паролей и атаки по словарю [39, 40, 41].

3.2.2 Авторизация через JWT

Авторизация пользователей реализована на базе JSON Web Token (JWT). При успешной проверке учётных данных `AuthService` формирует токен, подписанный секретным ключом `JWT_SECRET`, хранящимся в переменных окружения [15]. Полезная нагрузка токена содержит идентификатор пользователя, имя пользователя и роль, что позволяет серверу восстанавливать контекст пользователя без обращения к базе данных при каждом запросе. В коде проекта эта полезная нагрузка имеет поля `id`, `username` и `role`.

Защищённые маршруты API используют AuthGuard, который выполняет следующие действия [8]:

1. извлекает Bearer-токен из заголовка `Authorization`;
2. проверяет подпись токена через `JwtService.verifyAsync` с использованием секрета из `ConfigService`;
3. при успехе прикрепляет раскодированную полезную нагрузку к объекту запроса `request.user`, делая данные пользователя доступными в контроллерах;
4. при отсутствии или недействительности токена возбуждает `UnauthorizedException`, что приводит к ответу со статусом 401.

Подпись токена защищает его от подделки: любой изменённый токен не пройдёт проверку подписи. Срок действия токена задаётся при его генерации и ограничивает время, в течение которого украденный токен может быть использован.

3.2.3 HTTP-only cookies

Для хранения JWT-токена на стороне клиента используется HTTP-only cookie с именем `access_token`. Токен никогда не передаётся в JavaScript-код браузера: cookie устанавливается серверным Next.js API-маршрутом `POST /api/auth/login` после успешного получения токена от серверной части NestJS [42, 43]. При последующих запросах клиентский код Next.js не работает с токеном напрямую – вместо этого все обращения к серверной части идут через серверные маршруты `app/api/`, которые читают cookie из запроса и добавляют заголовок `Authorization: Bearer` перед перенаправлением запроса на серверную часть [44].

Такой подход обеспечивает защиту от нескольких типов атак. Во-первых, злоумышленник не может украсть токен через XSS-уязвимость: даже при внедрении произвольного JavaScript-кода на страницу он не сможет прочитать HTTP-only cookie. Во-вторых, клиентский код не содержит то-

кен в DOM, переменных или localStorage, что уменьшает поверхность атаки [45, 46].

Связь клиентского маршрута и серверной проверки токена приведена в приложении А: в листинге А.1 показан серверный маршрут Next.js, а в листинге А.2 – защитник AuthGuard. Серверный маршрут Next.js читает токен из HTTP-only cookie и добавляет его в заголовок Authorization. После этого защищённый маршрут серверной части проверяет токен через AuthGuard.

3.2.4 Ролевая модель доступа

В системе реализована ролевая модель с четырьмя ролями: USER, TEACHER, MODERATOR и ADMIN. Роль сохраняется в поле role таблицы User и передаётся в составе JWT-токена. Проверка роли выполняется при привилегированных операциях:

1. обычный пользователь может создавать публичные и личные публикации, редактировать и удалять собственные публикации, менять их режим доступа, добавлять доступные публикации в избранное, убирать их оттуда и ставить отметки «Нравится»;

2. обычный пользователь может отправить заявку на получение роли преподавателя, указав ФИО и загрузив изображение подтверждающего документа;

3. преподаватель имеет те же права, что и обычный пользователь, однако его публикации помечаются в интерфейсе как материалы преподавателя;

4. модератор дополнительно имеет право редактировать и удалять любые публикации для поддержания качества контента, обновлять логотипы университетов и одобрять заявки преподавателей;

5. администратор обладает полными правами на уровне пользовательского интерфейса, может работать с административными маршрутами модерации, редактировать и удалять любые публикации, расширять каталог, обновлять логотипы университетов и одобрять заявки преподавателей.

Клиентская часть также учитывает роль пользователя при отображении интерфейса: кнопки «Добавить» на страницах каталога показываются только администраторам, кнопка обновления логотипа университета – администраторам и модераторам, публикации преподавателей получают отдельную визуальную пометку, карточка перехода к заявке преподавателя размещена на странице «О продукте», а кнопки «Редактировать» и «Удалить» на странице публикации – только автору публикации или пользователю с ролью модератора или администратора. Раздел заявок преподавателей доступен только администраторам и модераторам. При этом серверные проверки остаются обязательными для операций с публикациями и административных маршрутов, чтобы пользователь не мог обойти ограничения прямым обращением к API. Для личных публикаций сервер дополнительно проверяет, что материал запрашивает автор или пользователь с привилегированной ролью.

3.2.5 Валидация входных данных

Все входные данные, поступающие от пользователя, проходят валидацию на сервере. Для полей формы проверяется наличие обязательных значений и соответствие типам; при нарушениях возбуждается `BadRequestException` со статусом 400. Для загружаемых файлов применяются дополнительные ограничения [47]:

1. проверка допустимости типа файла: изображения принимаются по MIME-типу, PDF-документы и MP4-видео – по MIME-типу и расширению; файлы с другими типами отклоняются на уровне `fileFilter` библиотеки `multer`;
2. ограничение количества файлов в одном запросе: до пяти вложений для публикации;
3. ограничение файла подтверждающего документа в заявке преподавателя: принимается ровно одно изображение;

4. ограничение количества публикаций: до десяти публикаций в день и до пятидесяти публикаций всего на одного пользователя;

5. генерация уникальных имён файлов по шаблону `{timestamp}-{random}.{ext}`, что исключает коллизии и возможность перезаписи чужих файлов.

Такая валидация предотвращает загрузку вредоносных файлов под видом разрешённых вложений, ограничивает злоупотребление массовой публикацией материалов и защищает файловую систему сервера от некорректных загрузок.

3.2.6 Защита от SQL-инъекций

Все запросы к базе данных формируются через Prisma Client, который использует параметризованные запросы. Значения полей передаются в виде аргументов запроса, а не конкатенируются в строку SQL, что исключает возможность внедрения постороннего SQL-кода через пользовательский ввод [48]. Декларативная схема базы данных и типобезопасный клиент обеспечивают дополнительный уровень защиты: невозможно сформировать некорректный запрос, используя поля, отсутствующие в модели [9].

3.2.7 Защита от XSS

Поскольку React по умолчанию экранирует все строковые значения при отображении, базовая защита от XSS-атак обеспечивается на уровне фреймворка [23]. Пользовательский текст: заголовки и тела публикаций, выводится в компонентах без использования `dangerouslySetInnerHTML`, что исключает интерпретацию HTML-кода из пользовательского ввода. Дополнительную защиту обеспечивает использование HTTP-only cookies для хранения токена [46].

3.2.8 Настройка CORS

Для ограничения возможных источников запросов к серверной части NestJS в приложении настроен CORS: разрешённые источники задаются переменной окружения `FRONTEND_ORIGIN`, а в среде разработки используется значение `http://localhost:3000`. Это предотвращает выполнение запросов к API из сторонних доменов и снижает риск межсайтовых атак в сочетании с использованием HTTP-only cookies [49].

Таким образом, в системе ConSuccess реализован комплекс мер информационной безопасности, охватывающих аутентификацию, авторизацию, валидацию входных данных, защиту от распространённых уязвимостей веб-приложений и ограничение поверхности атаки. Такой подход обеспечивает защиту как пользовательских данных, так и внутренней инфраструктуры системы.

3.3 Масштабируемость и перспективы развития

Текущая реализация веб-приложения ConSuccess покрывает ключевые сценарии обмена учебными материалами и обеспечивает работоспособность базового функционала: иерархический каталог, публикацию материалов с вложениями, раздел «Знания», личные публикации, авторизацию, механизм избранного, отметки «Нравится», заявки на роль преподавателя и контейнерный запуск приложения. Тем не менее, в перспективе развития платформы можно выделить несколько направлений, направленных на расширение возможностей, повышение производительности и улучшение пользовательского опыта.

3.3.1 Расширение каталога

Одним из первых шагов масштабирования является расширение базового классификатора. В текущей реализации каталог построен на иерархии

«университет – дисциплина – публикация». В дальнейшем могут быть добавлены следующие уровни и атрибуты:

1. Тип материала: конспект, лабораторная работа, задача, методические указания, пример кода, что позволит пользователям фильтровать контент не только по дисциплине, но и по характеру материала.

2. Теги – свободные метки, проставляемые автором публикации. Теги могут дополнять формальную классификацию и использоваться для семантического поиска.

3. Учебный семестр и год публикации – атрибуты, позволяющие отслеживать актуальность материалов и различать версии, относящиеся к разным периодам обучения.

4. Расширенная карточка преподавателя – дополнительные сведения о преподавателе, кафедре или группе, что полезно в случаях, когда одна и та же дисциплина ведётся разными преподавателями с разной программой. Эта карточка может развивать уже реализованный механизм подтверждения роли преподавателя.

Расширение схемы базы данных требует добавления новых таблиц и миграций Prisma, а также обновления форм публикации и страниц каталога. Поскольку Prisma поддерживает инкрементальные миграции, эти изменения могут быть внесены поэтапно, без нарушения работы существующей части системы.

3.3.2 Полнотекстовый поиск

В текущей реализации навигация по каталогу построена исключительно на иерархии «университет – дисциплина – публикация». Для большого каталога такой подход становится недостаточным: пользователь может знать ключевые слова искомого материала, но не помнить, к какой именно дисциплине он относится. Решением является внедрение полнотекстового поиска.

PostgreSQL предоставляет встроенные возможности полнотекстового поиска через тип данных `tsvector` и оператор `@@`. Для русского языка доступны словари-поисковики с поддержкой морфологии [50]. Интеграция в Prisma осуществляется через `unsupported` поля или прямые SQL-запросы через `prisma.$queryRaw`. Альтернативой является использование специализированных поисковых движков, таких как Meilisearch или Elasticsearch, которые обеспечивают индексацию и ранжирование результатов [51, 52].

Пользовательский интерфейс поиска может быть реализован в виде поисковой строки в шапке приложения, открывающей страницу результатов с фильтрами по университету, дисциплине, типу материала и тегам. Результаты поиска можно дополнительно ранжировать по популярности и дате публикации.

3.3.3 Модерация и система жалоб

С ростом количества пользователей увеличивается риск появления нежелательного контента: публикаций, нарушающих правила платформы, материалов с неточной информацией или материалов, защищённых авторским правом. В текущей реализации модерация осуществляется в ручном режиме пользователями с ролями MODERATOR и ADMIN, которые могут редактировать и удалять любые публикации. В дальнейшем предлагается внедрить систему жалоб:

1. кнопку «Пожаловаться» на странице публикации с возможностью выбора причины: некорректное содержимое, нарушение авторских прав, дубликат, спам;
2. очередь жалоб для модераторов с возможностью просмотра, принятия решения и обратной связи с автором публикации;
3. автоматическую блокировку публикации при превышении порога числа жалоб, с последующей проверкой модератором.

Данный механизм повышает прозрачность модерации и снижает нагрузку на модераторов, поскольку сообщество само участвует в выявлении проблемного контента.

3.3.4 Система рекомендаций

Для повышения вовлечённости пользователей может быть внедрена система рекомендаций. В простейшем варианте она может основываться на следующих принципах:

1. популярность: публикации с наибольшим числом отметок «Нравится» и добавлений в избранное выводятся на главной странице;
2. активность дисциплины: дисциплины с большим количеством свежих публикаций получают приоритет в списках;
3. коллаборативная фильтрация: пользователям, добавившим одни и те же публикации в избранное, показываются аналогичные материалы.

Реализация рекомендаций может использовать уже существующие данные об отметках «Нравится» и избранном, а также потребует ведения статистики просмотров и периодического пересчёта рекомендованных подборок.

3.3.5 Мобильная версия и адаптивность

В текущей реализации интерфейс поддерживает адаптивную вёрстку для мобильных экранов: страницы перестраиваются с использованием адаптивных классов Tailwind CSS, а навигация может отображаться в компактном мобильном варианте. Дальнейшее развитие предполагает дополнительную оптимизацию мобильного опыта с учётом особенностей сенсорных экранов: более крупные области нажатия, улучшение поведения галереи и доработку отдельных сценариев под небольшую ширину экрана.

3.3.6 Кэширование и производительность

По мере роста числа пользователей и объёма данных потребуется оптимизация производительности. Возможные направления:

1. Кэширование результатов запросов на стороне сервера через Redis. Часто запрашиваемые данные: список университетов, список дисциплин, могут храниться в кэше с ограниченным временем жизни, что снижает нагрузку на базу данных [53].

2. Клиентское кэширование часто запрашиваемых данных с настраиваемыми стратегиями обновления. Это уменьшает количество повторных запросов при навигации между страницами.

3. Оптимизация изображений: вместо отдачи оригинальных файлов возможна генерация миниатюр разных размеров и использование современных форматов WebP, AVIF. Для этого можно интегрировать библиотеку `sharp` на стороне сервера [54].

4. CDN для статических ресурсов: логотипы университетов, вложения публикаций, что ускоряет доставку контента географически удалённым пользователям.

3.3.7 Развёртывание и масштабирование инфраструктуры

Для продуктивной эксплуатации системы потребуется развитие уже подготовленной контейнерной конфигурации и переход от локального Docker-запуска к полноценному развёртыванию. Возможные варианты:

1. Использование подготовленных Docker-образов клиентской и серверной частей в среде непрерывной поставки, а также настройка отдельных профилей окружения для разработки и продуктивного запуска.

2. Размещение базы данных PostgreSQL на выделенном сервере или в облачном сервисе с настроенными резервными копиями и репликацией.

3. Вынесение хранилища загруженных файлов в объектное хранилище, например Amazon S3 или его совместимые аналоги, что делает серверную часть stateless и упрощает горизонтальное масштабирование [55].

4. Использование балансировщика нагрузки перед несколькими экземплярами серверной и клиентской частей для обеспечения отказоустойчивости и обработки возрастающего трафика.

3.3.8 Интеграция с внешними сервисами

Дальнейшая интеграция с внешними сервисами может расширить функциональность платформы. Например, поддержка входа через учётные записи социальных сетей или образовательных систем упростит регистрацию; интеграция с сервисами автоматического распознавания рукописного текста позволит преобразовывать фотографии конспектов в текст; добавление экспорта публикаций в формате PDF облегчит офлайн-использование материалов.

Таким образом, разработанная платформа обладает значительным потенциалом для дальнейшего развития. Модульная архитектура, типобезопасная работа с базой данных и чёткое разделение ответственности между клиентом и сервером создают основу для поэтапного расширения функциональности без необходимости переработки существующего кода. Перечисленные направления позволяют постепенно превратить ConSuccess из локального решения для обмена учебными материалами в полноценную образовательную платформу, поддерживающую широкий круг пользователей и разнообразные учебные сценарии.

3.4 Выводы по главе

В третьей главе была выполнена оценка разработанной системы через функциональные сценарии для основных пользовательских ролей, рассмотрены вопросы информационной безопасности и описаны направления мас-

штабирования. Проверка сценариев показала, что приложение поддерживает ключевые действия гостей, студентов, преподавателей, модераторов и администраторов, включая публикацию материалов, работу с личными записями, избранным и заявками на получение роли преподавателя. Рассмотренные меры защиты и перспективы развития подтверждают возможность дальнейшего практического использования и расширения платформы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было разработано веб-приложение ConSuccess, предназначенное для коллективного обмена учебными материалами между студентами и преподавателями. Система решает актуальную задачу фрагментации учебных ресурсов, с которой сталкиваются студенты при подготовке к занятиям: неформальные каналы обмена, такие как мессенджеры, облачные диски и личные папки, не обеспечивают единой структуры хранения, удобной навигации и механизмов повторного доступа к найденным материалам. Разработанная платформа предоставляет иерархический каталог «университет – дисциплина – публикация» с поддержкой текстовых публикаций, прикрепления вложений разных типов, раздела «Знания» для общих материалов, личных публикаций, пометки материалов преподавателя, механизма избранного для формирования персональной базы знаний и проверяемых заявок на получение роли преподавателя.

В рамках работы были решены задачи, поставленные во введении:

1. Провести анализ предметной области и выявить проблемы существующих способов обмена учебными материалами.
2. Провести обзор аналогов систем обмена учебными материалами и определить их преимущества и недостатки.
3. Определить целевую аудиторию веб-приложения и пользовательские роли.
4. Сформировать функциональные и нефункциональные требования к системе.
5. Спроектировать структуру и навигацию пользовательского интерфейса, подготовить макеты ключевых экранов.
6. Спроектировать архитектуру веб-приложения и клиент-серверное взаимодействие.
7. Разработать модель данных и спроектировать схему базы данных.
8. Реализовать серверную часть веб-приложения.

9. Реализовать клиентскую часть веб-приложения.
10. Провести тестирование разработанной системы и устранить выявленные недостатки.
11. Обеспечить и описать меры информационной безопасности веб-приложения.
12. Описать возможности масштабирования и дальнейшего развития системы.

Разработанная система обеспечивает решение поставленной цели – создание централизованной платформы для коллективного обмена учебными материалами, организованной по учебным заведениям и дисциплинам. Применение современных технологий, таких как Next.js, NestJS, PostgreSQL, Prisma и Tailwind CSS, обеспечивает производительность, типобезопасность и удобство дальнейшей поддержки кодовой базы. Модульная архитектура и чёткое разделение серверного и клиентского слоёв создают основу для поэтапного расширения функциональности без необходимости переработки существующего кода.

Практическая значимость работы заключается в возможности использования разработанной платформы в студенческих сообществах вузов: система упорядочивает разрозненные учебные материалы, упрощает навигацию к ним, обеспечивает безопасный доступ и формирует персональные подборки полезного контента. По сравнению с неформальными каналами обмена и существующими аналогами, ConSuccess сочетает в себе иерархический каталог, поддержку вложений разных типов, личные материалы, выделение публикаций преподавателя, проверку получения роли преподавателя и механизм избранного, что делает её самодостаточным решением для учебной среды.

Перспективы дальнейшего развития включают внедрение полнотекстового поиска, расширение классификаторов: тип материала, теги, семестр, преподаватель, системы жалоб и модерации, интеграции с внешними сервисами, расширение профиля подтверждённого преподавателя и дальнейшую оптимизацию мобильного интерфейса. Эти направления позволяют посте-

пенно превратить систему из локального решения в полноценную образовательную платформу, поддерживающую широкий круг пользователей и разнообразные учебные сценарии.

Таким образом, все поставленные задачи выполнены, цель работы достигнута. Разработанное веб-приложение готово к использованию в учебном сообществе и может служить основой для дальнейших исследований и практических проектов в области цифрового образования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Федеральный закон от 29.12.2012 № 273-ФЗ «Об образовании в Российской Федерации». – <https://base.garant.ru/70291362/>. – Дата обращения: 10.05.2026.
2. Fowler, Martin. Patterns of Enterprise Application Architecture / Martin Fowler. – Addison-Wesley Professional, 2003. – P. 560.
3. Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design / Robert C. Martin. – Prentice Hall, 2017. – P. 432.
4. Kleppmann, Martin. Designing Data-Intensive Applications / Martin Kleppmann. – O'Reilly Media, 2017. – P. 616.
5. NestJS – A progressive Node.js framework. Официальная документация. – <https://docs.nestjs.com/>. – Дата обращения: 01.04.2026.
6. Node.js Documentation. – <https://nodejs.org/api/>. – Дата обращения: 10.05.2026.
7. TypeScript Handbook. – <https://www.typescriptlang.org/docs/>. – Дата обращения: 03.04.2026.
8. NestJS Documentation. Guards. – <https://docs.nestjs.com/guards>. – Дата обращения: 10.05.2026.
9. Prisma ORM Documentation. Schema, migrations, client. – <https://www.prisma.io/docs>. – Дата обращения: 02.04.2026.
10. Prisma ORM Documentation. Relations. – <https://www.prisma.io/docs/orm/prisma-schema/data-model/relations>. – Дата обращения: 10.05.2026.
11. Prisma ORM Documentation. Prisma Migrate. – <https://www.prisma.io/docs/orm/prisma-migrate>. – Дата обращения: 10.05.2026.
12. PostgreSQL 16 Documentation. – <https://www.postgresql.org/docs/16/>. – Дата обращения: 02.04.2026.
13. PostgreSQL Documentation. Indexes. – <https://www.postgresql.org/docs/current/indexes.html>. – Дата обращения: 10.05.2026.

14. Грофф, Дж. SQL. Полное руководство / Дж. Грофф, П. Вайнберг. – 3 edition. – М.: Вильямс, 2019. – Р. 960.
15. JSON Web Tokens Introduction (RFC 7519). – <https://datatracker.ietf.org/doc/html/rfc7519>. – Дата обращения: 06.04.2026.
16. NestJS Documentation. Authentication. – <https://docs.nestjs.com/security/authentication>. – Дата обращения: 10.05.2026.
17. MDN Web Docs. Authorization header. – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Authorization>. – Дата обращения: 10.05.2026.
18. RFC 9110. HTTP Semantics. – <https://www.rfc-editor.org/rfc/rfc9110>. – Дата обращения: 10.05.2026.
19. Multer – Node.js middleware for handling multipart/form-data. – <https://github.com/expressjs/multer>. – Дата обращения: 06.04.2026.
20. NestJS Documentation. File upload. – <https://docs.nestjs.com/techniques/file-upload>. – Дата обращения: 10.05.2026.
21. Next.js Documentation. App Router. – <https://nextjs.org/docs/app>. – Дата обращения: 01.04.2026.
22. Next.js Documentation. Server and Client Components. – <https://nextjs.org/docs/app/getting-started/server-and-client-components>. – Дата обращения: 10.05.2026.
23. React – The library for web and native user interfaces. Официальная документация. – <https://react.dev/>. – Дата обращения: 03.04.2026.
24. Next.js Documentation. Route Handlers. – <https://nextjs.org/docs/app/api-reference/file-conventions/route>. – Дата обращения: 10.05.2026.
25. Tailwind CSS v4 Documentation. – <https://tailwindcss.com/docs>. – Дата обращения: 04.04.2026.
26. Axios – Promise based HTTP client for the browser and node.js. – <https://axios-http.com/docs/intro>. – Дата обращения: 07.04.2026.
27. React Hook Form Documentation. – <https://react-hook-form.com/docs>. – Дата обращения: 05.04.2026.

28. Next.js Documentation. revalidatePath. – <https://nextjs.org/docs/app/api-reference/functions/revalidatePath>. – Дата обращения: 10.05.2026.
29. Next.js Documentation. Image Component. – <https://nextjs.org/docs/app/api-reference/components/image>. – Дата обращения: 10.05.2026.
30. Osmani, Addy. Learning JavaScript Design Patterns / Addy Osmani. – O'Reilly Media, 2023. – P. 304.
31. ГОСТ Р ИСО/МЭК 25010-2015. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов. – <https://protect.gost.ru/default.aspx/v.aspx?control=7&id=200427>. – Дата обращения: 10.05.2026.
32. Vitest Documentation. Getting Started. – <https://vitest.dev/guide/>. – Дата обращения: 10.05.2026.
33. Testing Library Documentation. React Testing Library. – <https://testing-library.com/docs/react-testing-library/intro/>. – Дата обращения: 10.05.2026.
34. Jest Documentation. Getting Started. – <https://jestjs.io/docs/getting-started>. – Дата обращения: 10.05.2026.
35. OWASP Top Ten Project – Top 10 Web Application Security Risks. – <https://owasp.org/www-project-top-ten/>. – Дата обращения: 08.04.2026.
36. ГОСТ Р 56939-2016. Защита информации. Разработка безопасного программного обеспечения. Общие требования. – <https://docs.cntd.ru/document/1200135525>. – Дата обращения: 10.05.2026.
37. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации». – <https://base.garant.ru/12148555/>. – Дата обращения: 10.05.2026.
38. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных». – <https://base.garant.ru/12148567/>. – Дата обращения: 10.05.2026.
39. bcrypt – A library to help you hash passwords. – <https://github.com/kellevin/node.bcrypt.js>. – Дата обращения: 07.04.2026.
40. NIST Special Publication 800-63B. Digital Identity Guidelines: Authentication and Lifecycle Management. – <https://pages.nist.gov/800-63-4/sp800-63b>

.html. – Дата обращения: 10.05.2026.

41. OWASP Cheat Sheet Series. Authentication Cheat Sheet. – https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. – Дата обращения: 10.05.2026.

42. MDN Web Docs. HTTP Cookies. – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>. – Дата обращения: 08.04.2026.

43. RFC 6265. HTTP State Management Mechanism. – <https://www.rfc-editor.org/rfc/rfc6265>. – Дата обращения: 10.05.2026.

44. OWASP Cheat Sheet Series. Session Management Cheat Sheet. – https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. – Дата обращения: 10.05.2026.

45. MDN Web Docs. Cross-site scripting (XSS). – <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>. – Дата обращения: 10.05.2026.

46. OWASP Cheat Sheet Series. Cross Site Scripting Prevention Cheat Sheet. – https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html. – Дата обращения: 10.05.2026.

47. OWASP Cheat Sheet Series. File Upload Cheat Sheet. – https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html. – Дата обращения: 10.05.2026.

48. OWASP Cheat Sheet Series. SQL Injection Prevention Cheat Sheet. – https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. – Дата обращения: 10.05.2026.

49. MDN Web Docs. Cross-Origin Resource Sharing (CORS). – <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>. – Дата обращения: 10.05.2026.

50. PostgreSQL Documentation. Full Text Search. – <https://www.postgresql.org/docs/current/textsearch.html>. – Дата обращения: 10.05.2026.

51. Meilisearch Documentation. – https://www.meilisearch.com/docs/getting_started/overview. – Дата обращения: 10.05.2026.

52. Elasticsearch Reference. – <https://www.elastic.co/docs/reference/elasticsearch>. – Дата обращения: 10.05.2026.
53. Redis Documentation. – <https://redis.io/docs/latest/>. – Дата обращения: 10.05.2026.
54. sharp Documentation. High performance Node.js image processing. – <https://sharp.pixelplumbing.com/>. – Дата обращения: 10.05.2026.
55. Newman, Sam. Building Microservices: Designing Fine-Grained Systems / Sam Newman. – O'Reilly Media, 2021. – P. 612.
56. Docker Documentation. What is Docker? – <https://docs.docker.com/get-started/docker-overview/>. – Дата обращения: 10.05.2026.

ПРИЛОЖЕНИЕ А

Листинги программного кода

Листинг А.1 – Передача JWT-токена из серверного маршрута Next.js

```
1 export async function POST(  
2   req: Request,  
3   { params }: { params: Promise<{ subjectId: string }> },  
4 ) {  
5   const cookieStore = await cookies();  
6   const accessToken = cookieStore.get('access_token')?.value;  
  
7   if (!accessToken) {  
8     return NextResponse.json({ message: 'Unauthorized' }, { status:  
9       ↪ 401 });  
10  }  
  
11  const { subjectId } = await params;  
12  const contentType = req.headers.get('content-type') ?? '';  
13  const body = Buffer.from(await req.arrayBuffer());  
  
14  const { data } = await axiosApi.post(  
15    `${endpoints.subjects}/${subjectId}/posts`,  
16    body,  
17    {  
18      headers: {  
19        Authorization: `Bearer ${accessToken}`,  
20        'Content-Type': contentType,  
21      },  
22    });  
  
23  revalidatePath('/universities', 'layout');  
24  return NextResponse.json(data, { status: 201 });  
25 }
```

Листинг А.2 – Проверка JWT-токена в серверном защитнике AuthGuard

```
1 @Injectable()
2 export class AuthGuard implements CanActivate {
3   constructor(private jwtService: JwtService) {}
4
5   async canActivate(context: ExecutionContext): Promise<boolean> {
6     const request = context.switchToHttp().getRequest();
7     const token = this.extractTokenFromHeader(request);
8
9     if (!token) {
10       throw new UnauthorizedException('No token provided');
11     }
12     try {
13       const payload = await
14         ↪ this.jwtService.verifyAsync<TokenPayload>(token);
15       request['user'] = payload;
16     } catch {
17       throw new UnauthorizedException('Invalid token');
18     }
19     return true;
20   }
21
22   private extractTokenFromHeader(request: Request): string |
23     ↪ undefined {
24     const [type, token] = request.headers.authorization?.split(' ')
25     ↪ ?? [];
26     return type === 'Bearer' ? token : undefined;
27   }
28 }
```